



AHEAD Online

Data Structures and Algorithms

Geetha Lekshmy V

Department of Computer Science

Amrita Vishwa Vidyapeetham

Objectives

Graph Data Structure

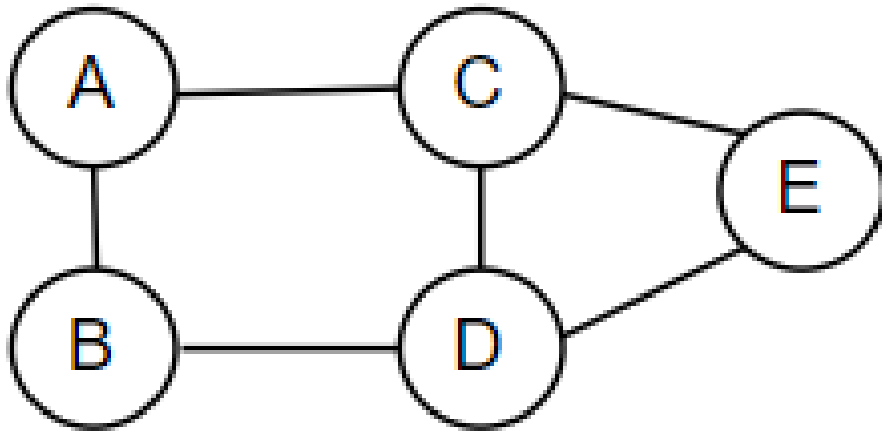
Definitions and Terminologies

Types of Graphs



Graph

- A Graph consists of two sets V, E
 - Vertices, V – finite non-empty set of nodes.
 - Edges or Arcs, E – pairs of vertices.



Graph, $G=(V,E)$

$V(G)=\{A,B,C,D,E\}$

$E(G)=\{(A,B), (B,D), (A,C), (D,C), (C,E), (D,E)\}$

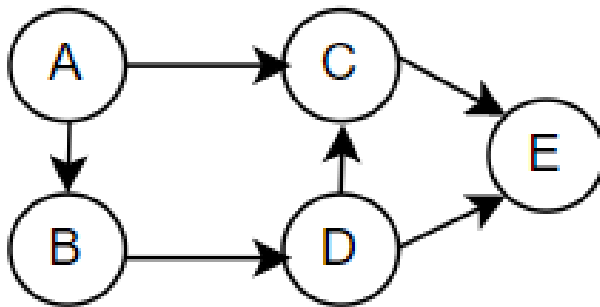
Adjacent Vertices -When an edge connects two vertices, they are said to be adjacent.
vertex A and B are adjacent

Directed Graph (Digraph)

- Directed Edge

A directed edge (u, v) is an ordered pair of vertices u, v where u precedes v . u is the origin and v is the destination.

- Directed Graph - all edges are directed.

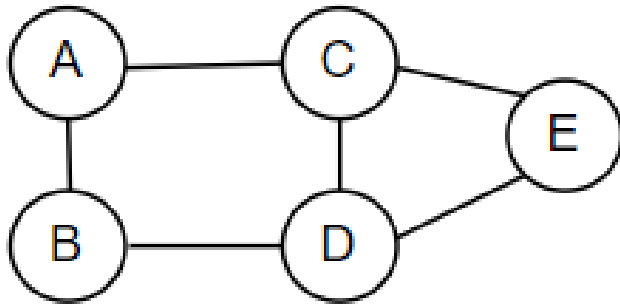


Undirected Graph

- Undirected Edge

An undirected edge (u,v) is an unordered pair of vertices (u,v) .

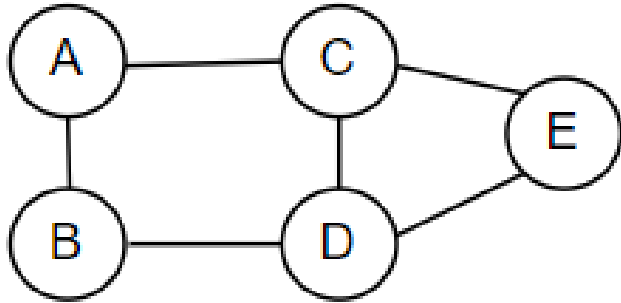
- Undirected Graph - all edges are undirected.



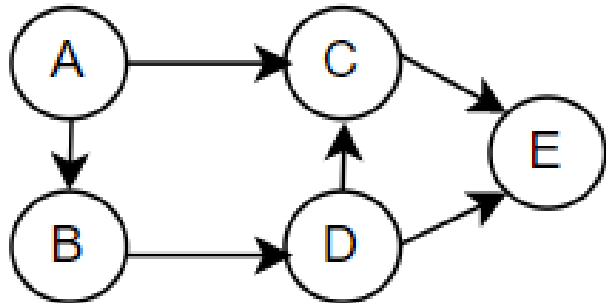
AMRITA | AHEAD
HWA VIDYAPEETHAM | Online

Graph

Degree of a vertex - Number of edges incident to it.



Degree of vertex A = 2, vertex D = 3

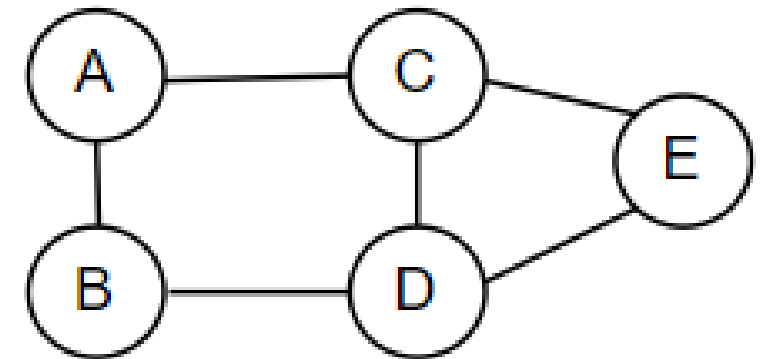


In-degree of vertex C - 2 (no. of edges with its head part in C)

Out-degree of vertex C - 1 (no. of edges with its tail part in C)

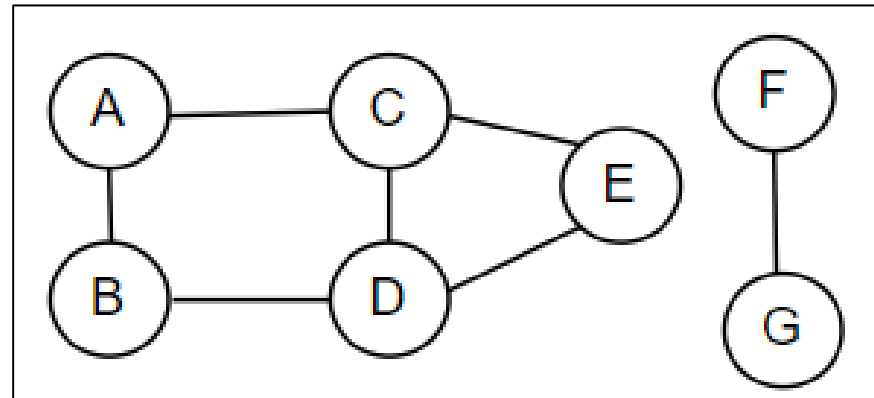
Graph

- Path – sequence of adjacent vertices.
- Length of the path is the number of edges on it.
- Simple path – a path in which all vertices except possibly the first and last are distinct.
- Path from A to E is a sequence of vertices A, B, D, E such that (A,B), (B,D) and (D,E) are edges in $E(G)$. It is a simple path with a length 3.
- Path A, C, D, E, C, D is **not** a simple path.
- A, B, C, D is not a path, (B,C) is not in $E(G)$.



Graph

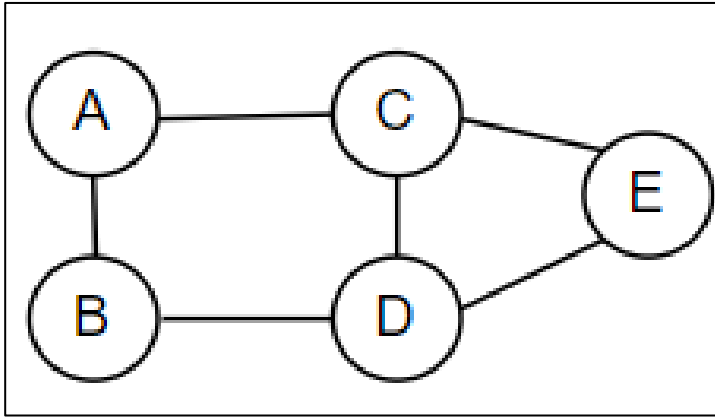
- Cycle – a simple path in which first and last vertices are the same.
- In the given graph, A,B,D,C,A is a cycle.
- In an undirected graph, G two vertices A and B are said to be connected if there is a path in G from A to B.
- Vertices A and E are connected but vertices A and F are not connected.



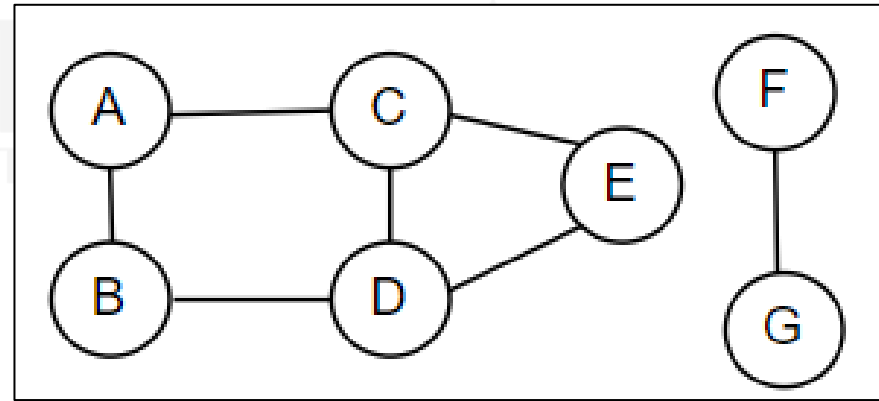
Graph

- An undirected graph G is said to be **connected** if for every pair of distinct vertices v_i, v_j in $V(G)$ there is a path from v_i to v_j in G .

A connected graph



A graph that is not connected, but with connected components.

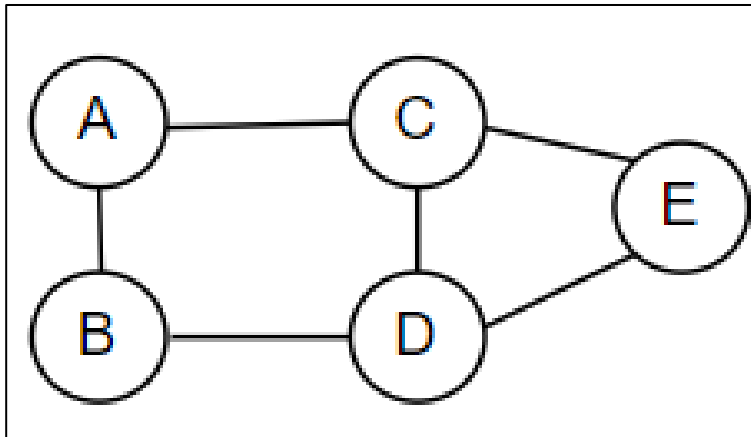


Connected component - any subgraph in which each pair of nodes is connected via a path.

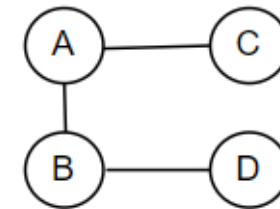
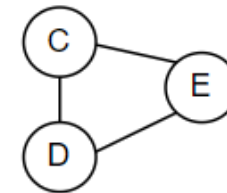
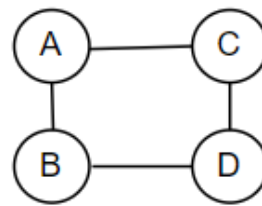
Subgraph

- Subgraph H , of a graph G , will have $V(H) \subseteq V(G)$ and $E(H) \subseteq E(G)$

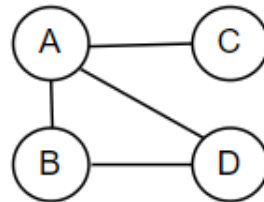
Graph G



Some Subgraphs of G



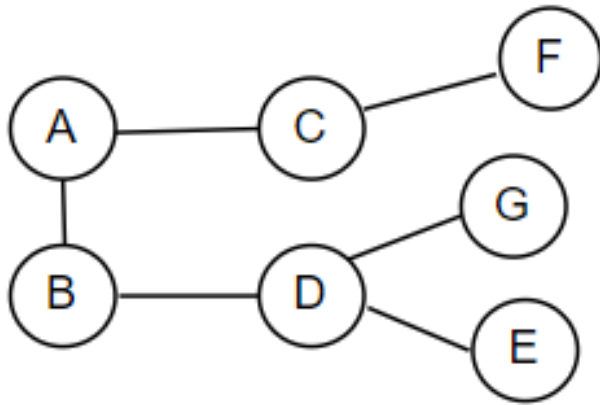
A graph that is not a subgraphs of G



Edge AD is not in G

Graph

- Is this a graph? Yes, An acyclic graph -Tree

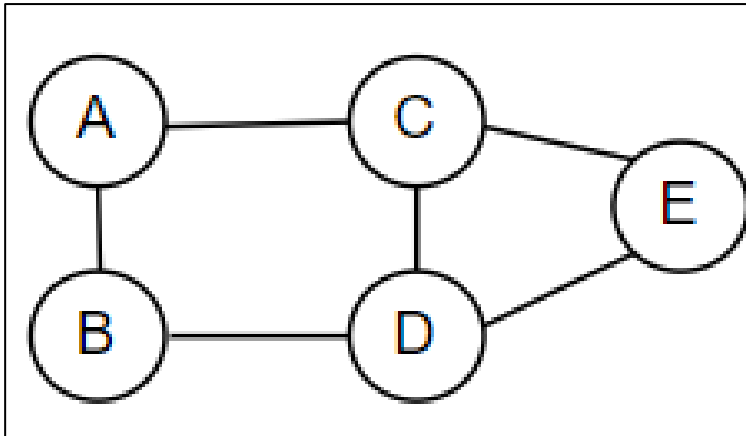


AMRITA | AHEAD
VIDYAPEETHAM | Online

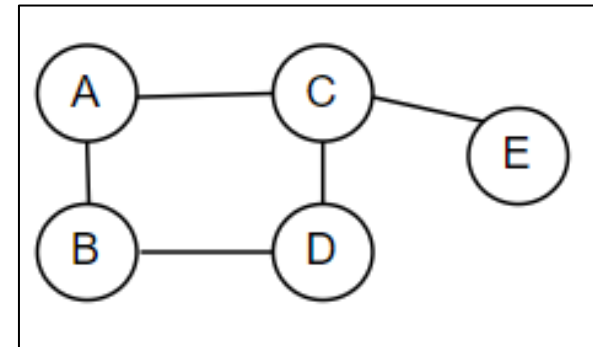
Spanning Subgraph

A spanning subgraph of G is a subgraph of G that contains all the vertices of the graph G .

Graph G

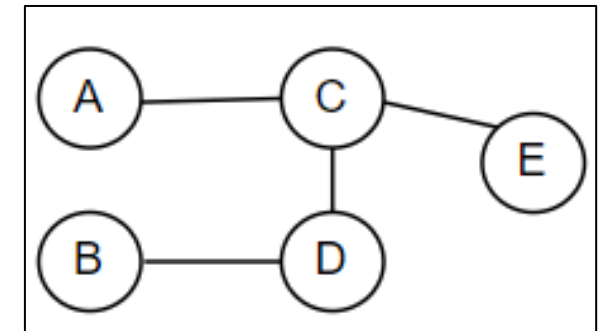
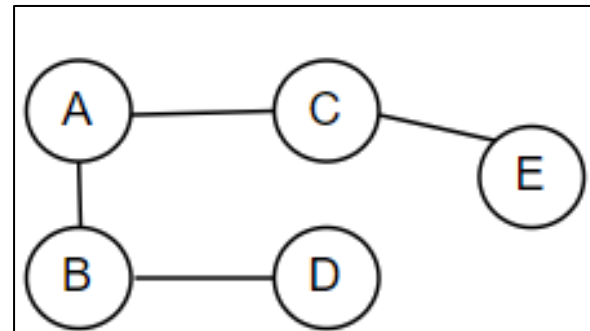


Spanning subgraphs



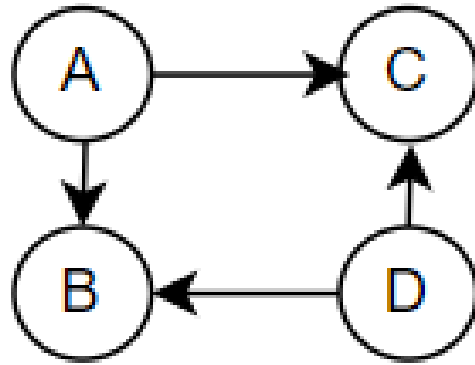
Spanning
Tree

Spanning
Tree



Directed Acyclic Graph(DAG)

DAG - directed graph with no cycles.

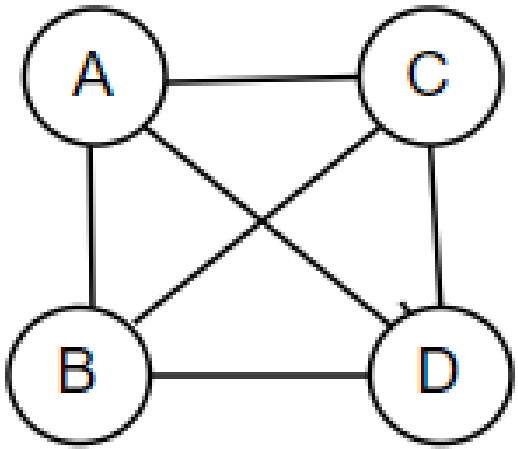


AHEAD
HAM | Online



Complete Graph

Complete graph - undirected graph in which each pair of graph vertices is connected by an **edge**.



Graph - Applications

- Representation of information on Social Networking platforms.(Facebook, Instagram, Linked in etc.)
- Google Maps - Find shortest path.
- Circuit Design.
- Routing Packets in a network.
- Google's Knowledge Graph

Summary

Graph

Non linear data structure

Widely used in diverse applications.



References

Data Structures and Algorithms in Java - Michael T. Goodrich
Data Structures - Ellis Horowitz, Sartaj Sahni

<https://developers.facebook.com/docs/graph-api/overview>
<https://developers.google.com/knowledge-graph>



Objectives

Graph – Traversal

Breadth First Search



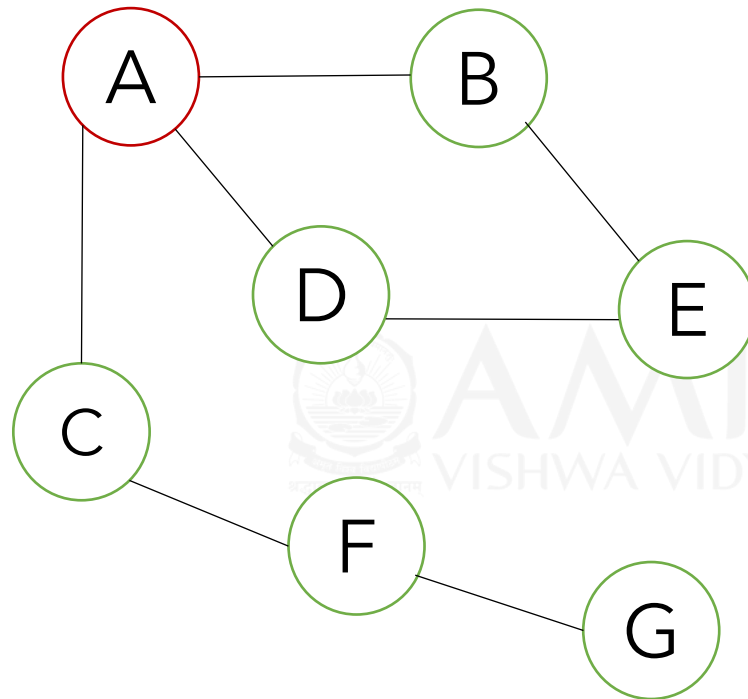
Graph Traversal

Breadth-First Search (BFS)

- Given a graph $G=(V,E)$ and a source vertex **s**, BFS explores the edges to find out all vertices that are reachable from **s**.
- Computes distance from **s** to each reachable vertex.
- It produces a breadth-first tree with **s** as root, and contains all reachable nodes from **s**.
- The path from **s** to a vertex v in breadth-first tree will be the shortest path from **s** to v

Breadth-First Search

Source vertex



Queue

A	
---	--

Visited

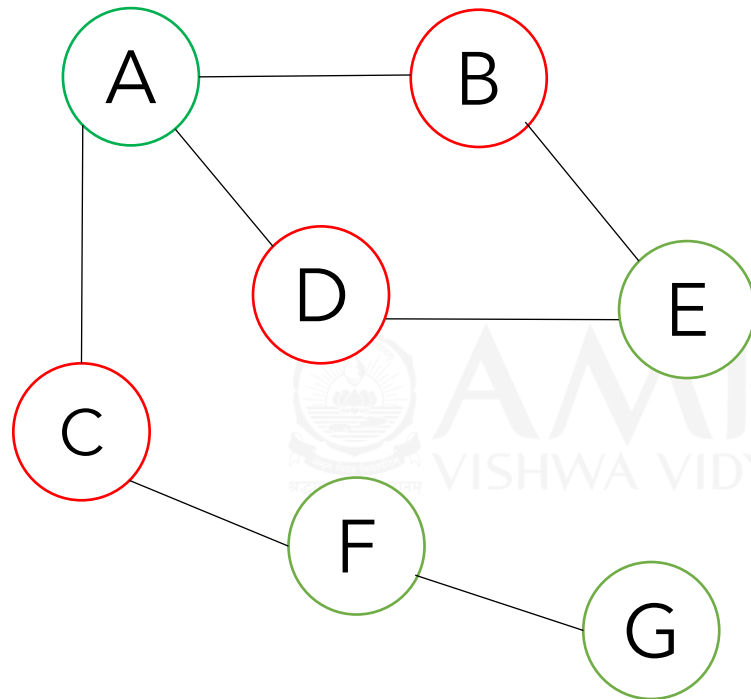
0 ₁	0	0	0	0	0	0
----------------	---	---	---	---	---	---

Output

A

Breadth-First Search

Source vertex



Queue

B	C	D	E	F
---	---	---	---	---

Visited

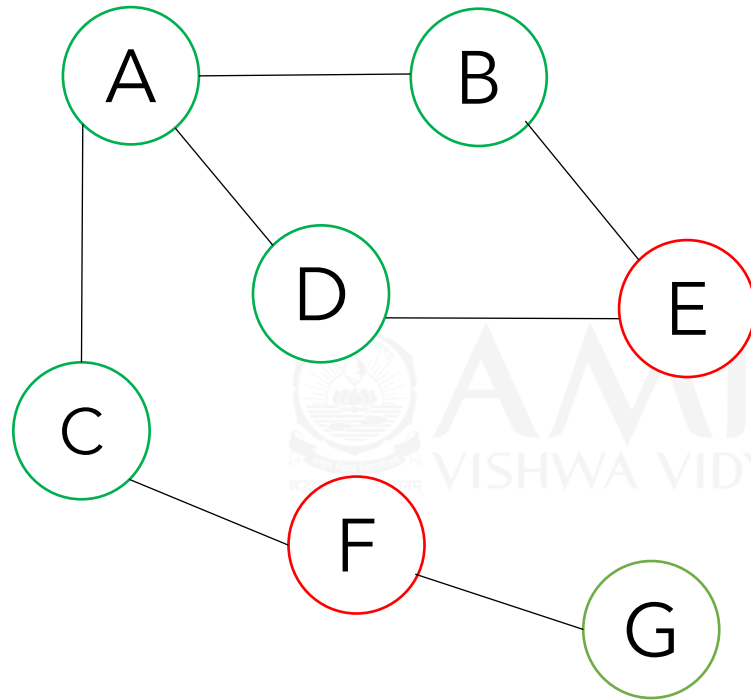
1	0	1	1	0	0	0
---	---	---	---	---	---	---

Output

A B C D

Breadth-First Search

Source vertex



Queue

E	F	G	
---	---	---	--

Visited

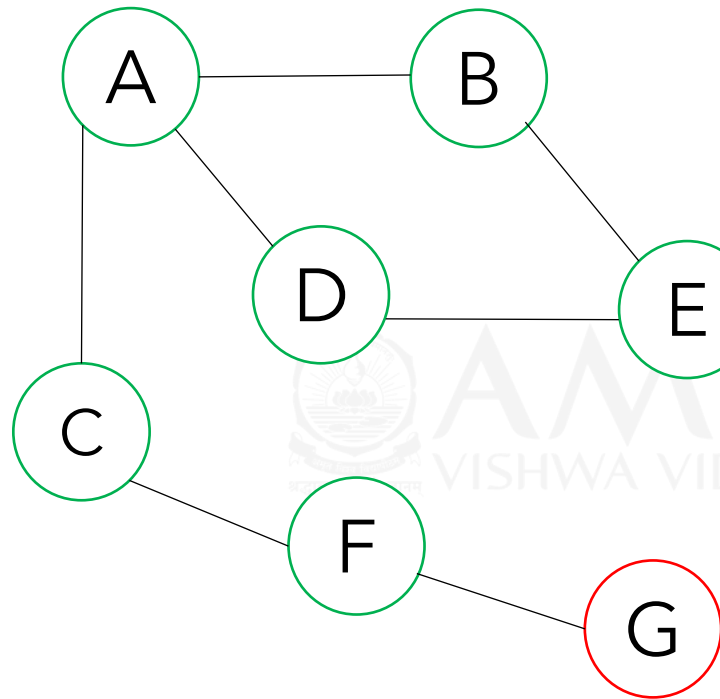
1	1	1	1	1	0	1	0	0
---	---	---	---	---	---	---	---	---

Output

A B C D E F

Breadth-First Search

Source vertex , s



Queue

G			
---	--	--	--

Visited

1	1	1	1	1	1	1 0
---	---	---	---	---	---	-----

Output

A B C D E F G

Breadth-First Search

```
BFS(Graph g, int source){  
    int visited[n] //n=no. of nodes in g  
    Queue Q  
    Q.insert(source)  
    while(!Q.isEmpty()){  
        v1=Q.dequeue();  
        print(v1)  
        visited[v1] = True  
        while (there are unvisited adjacent node v2 for v1 in g)  
        {  
            Q.insert(v2)  
        }  
    }  
}
```

Summary

Breadth First Search



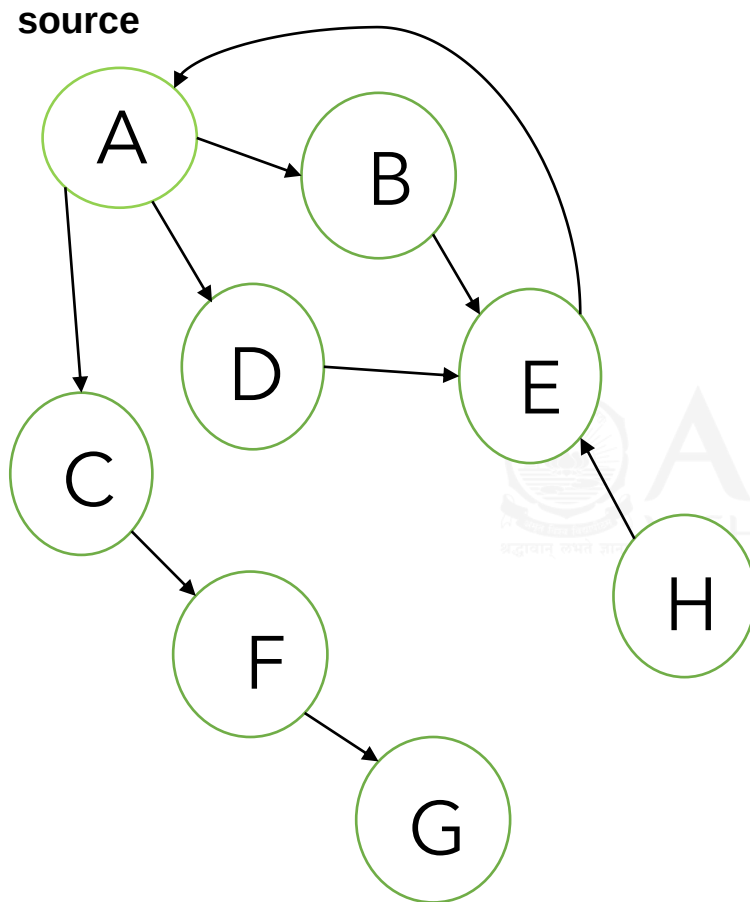
Objectives

Graph – Traversal

Depth First Search

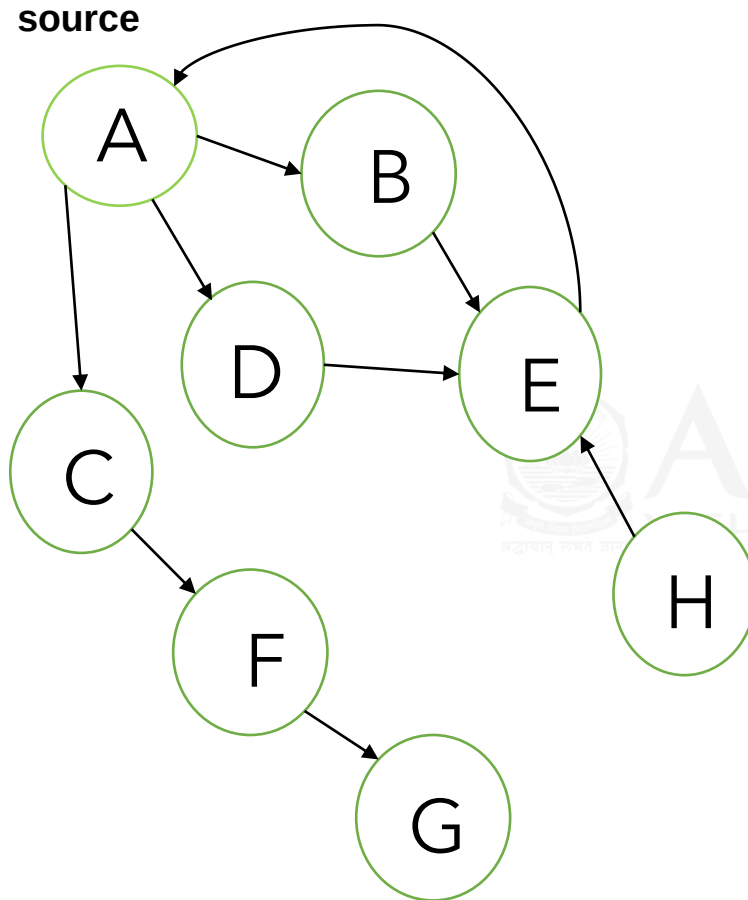


Depth First Search



- Search deeper in the graph.
- Colours vertices to indicate its state.
 - ✓ White - initially
 - ✓ Grey - discovered
 - ✓ Black - finished processing
- Each node, v has a two time stamps
 - ✓ $v.d$ = time when v is discovered.
 - ✓ $v.f$ = time when search finishes examining v 's adjacency list.

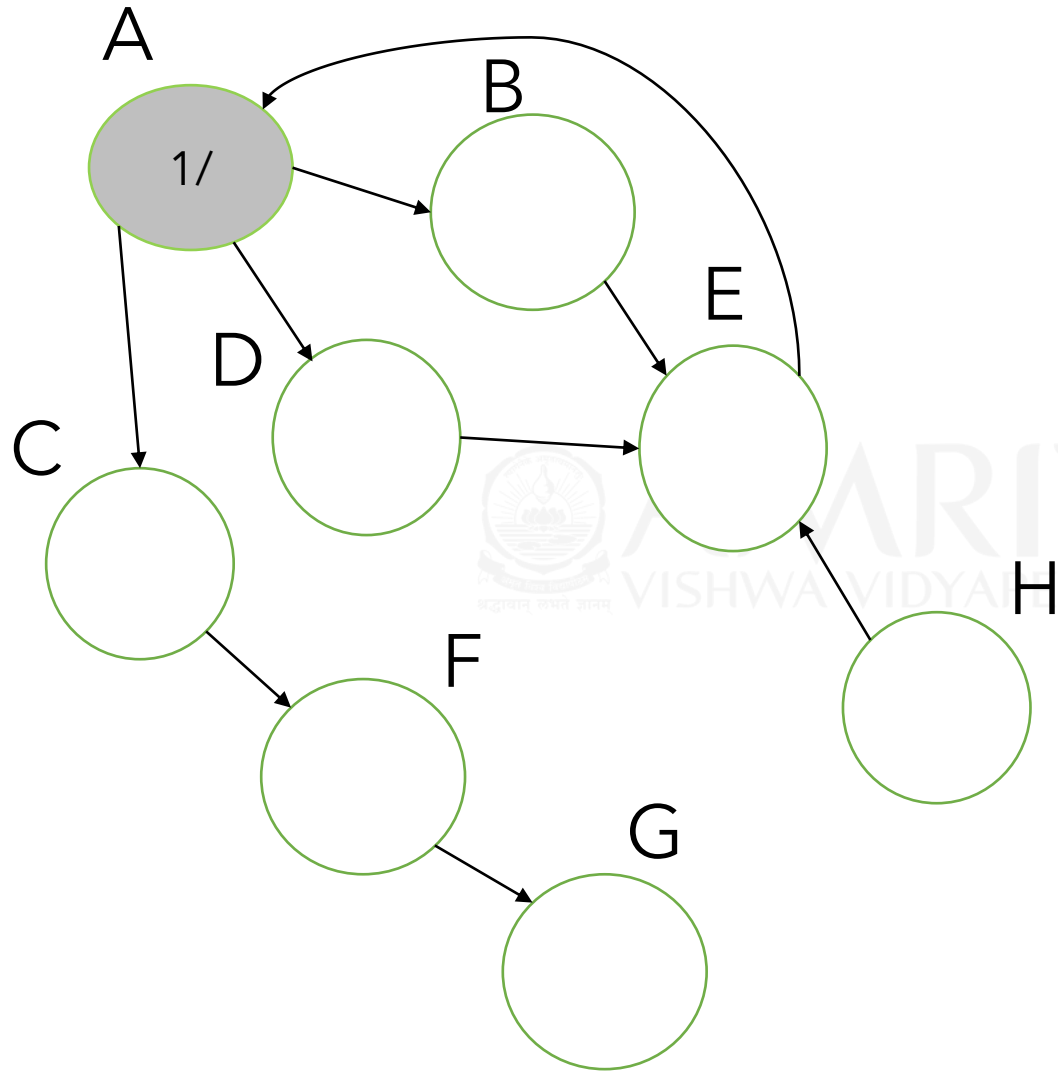
Depth First Search



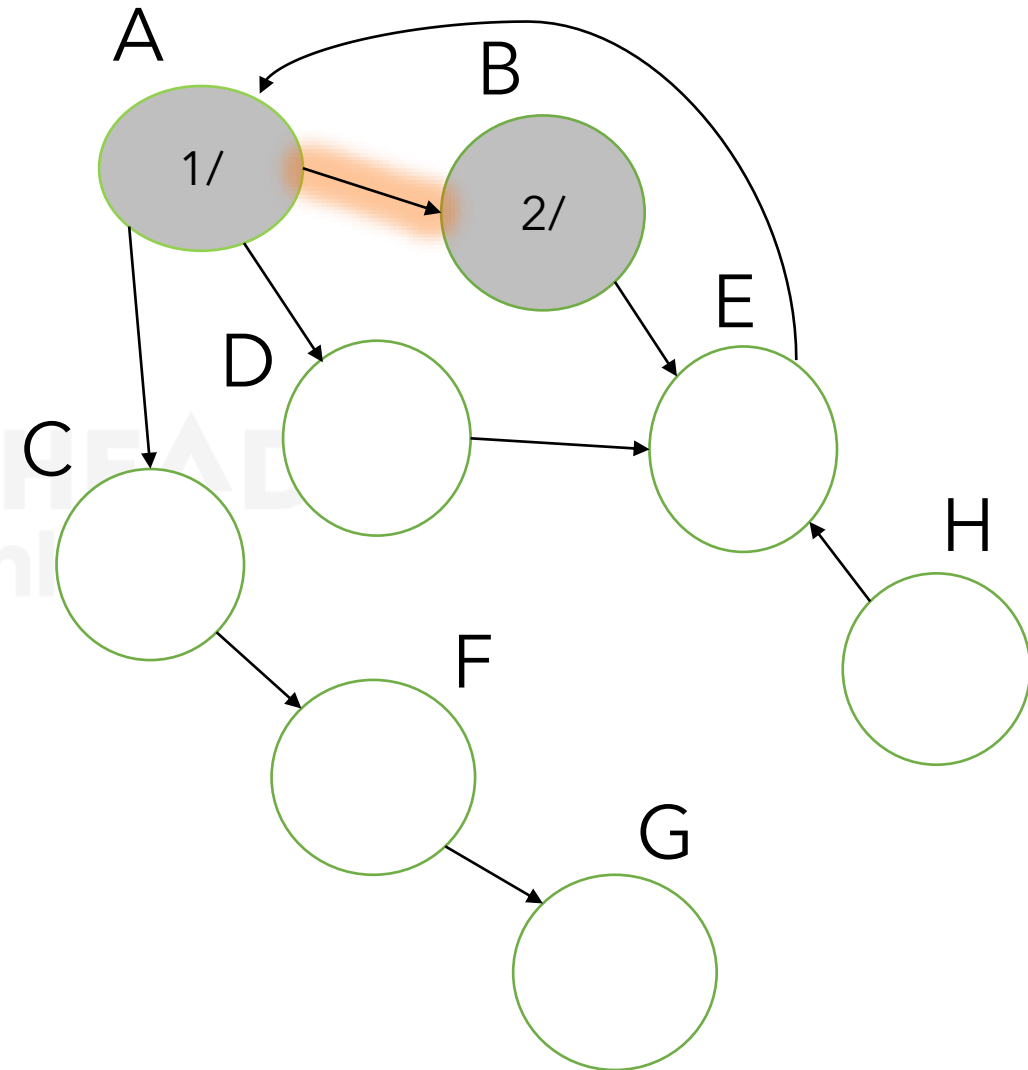
- Predecessor attribute of a node v , is set to u , if v is discovered by scanning vertices of u .
- Predecessor subgraph - depth first forest with depth first trees.

Depth First Search

T = 1

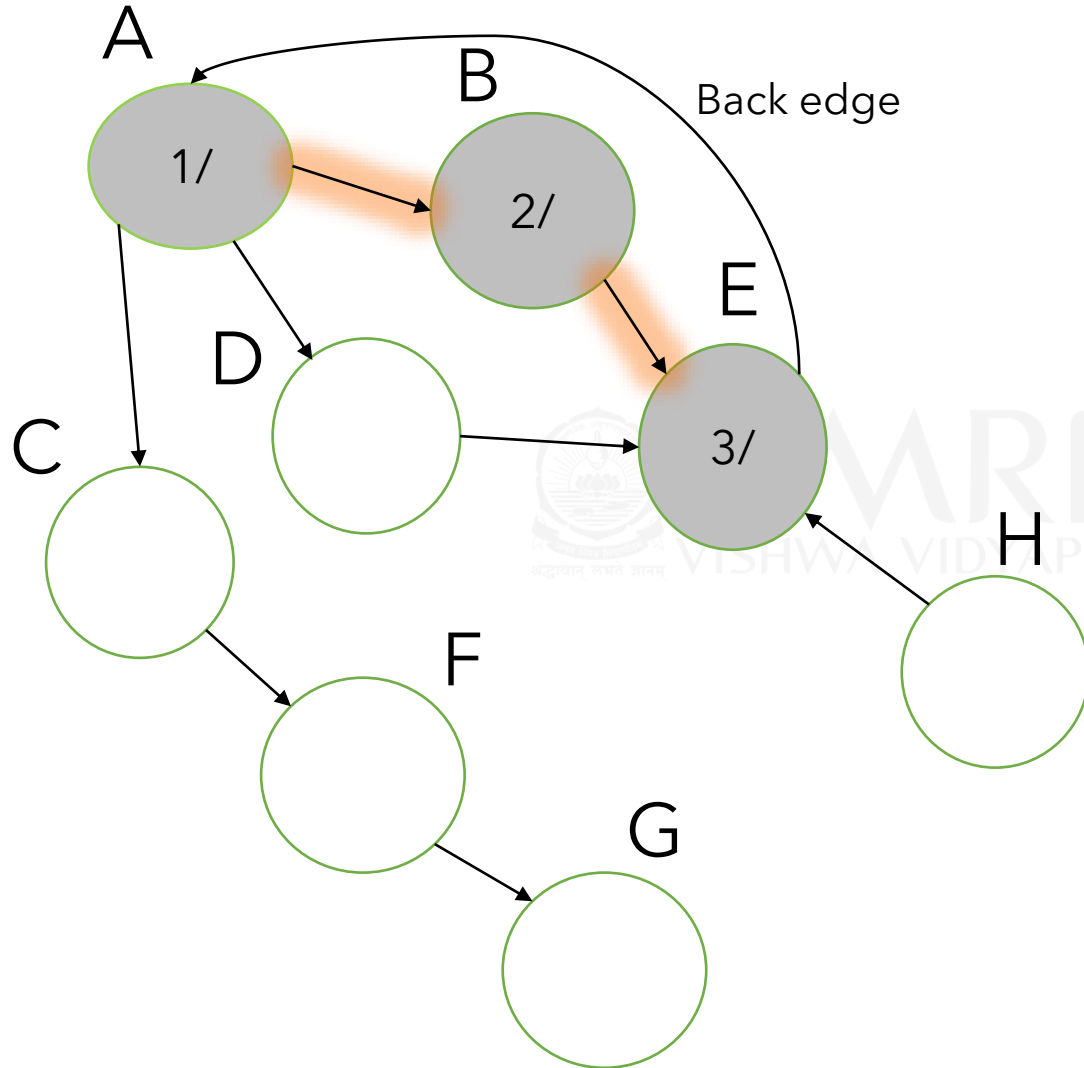


T = 2

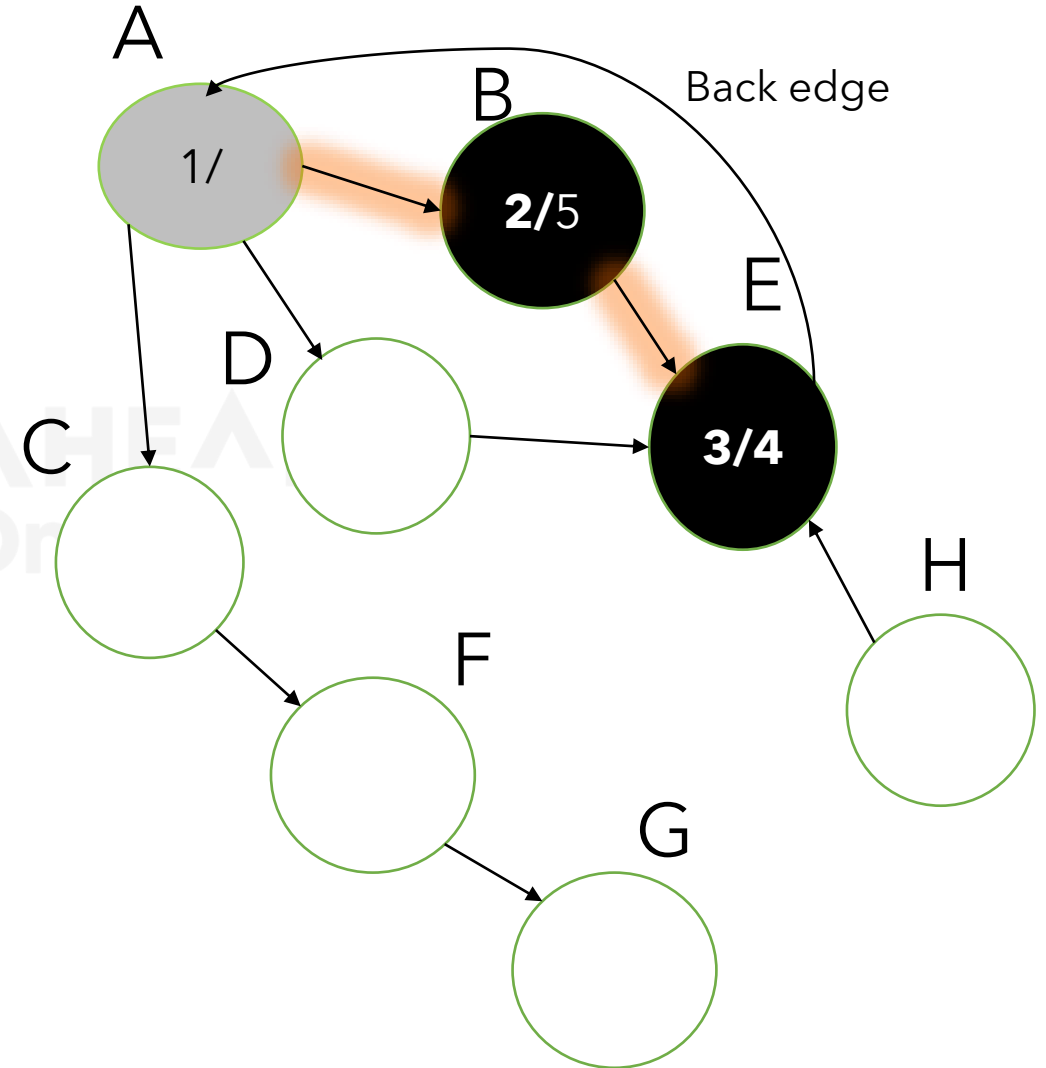


Depth First Search

T = 3

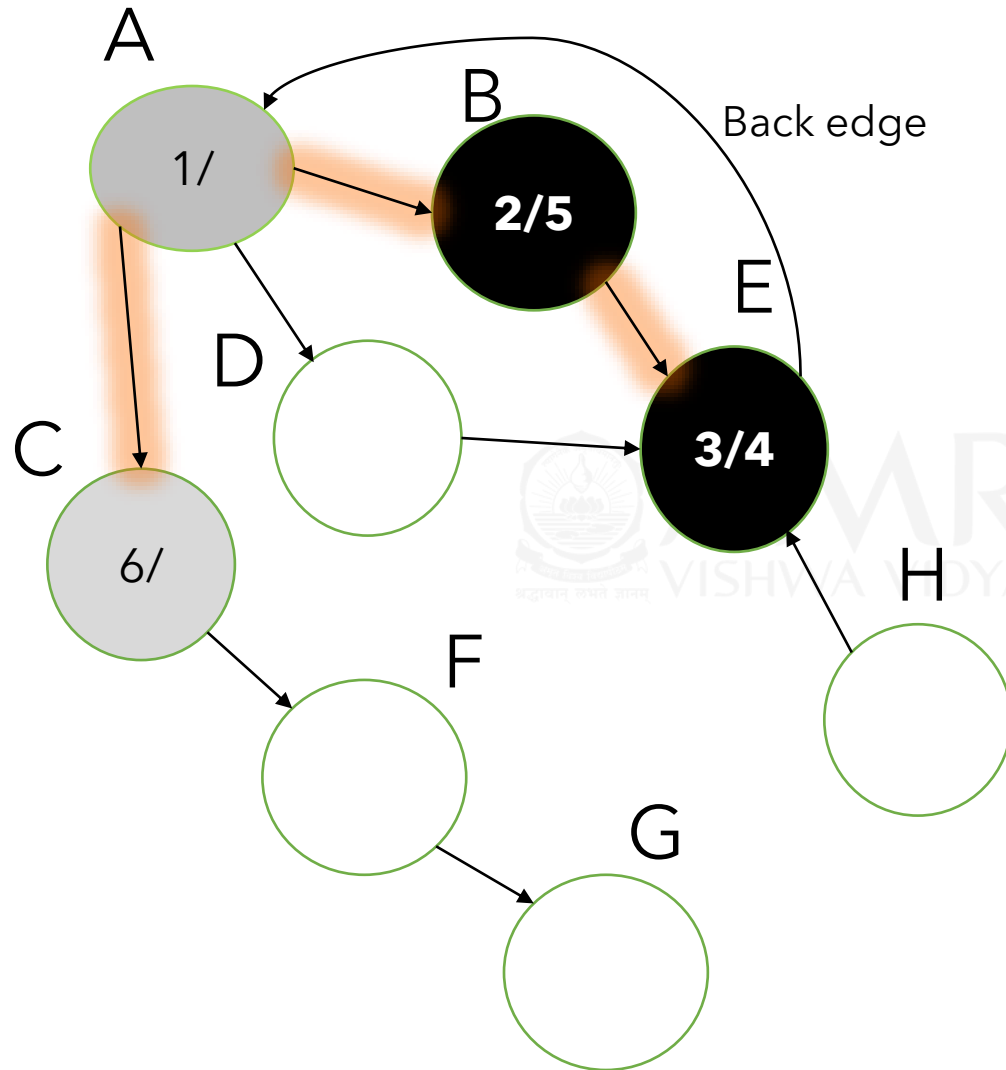


T = 4, 5

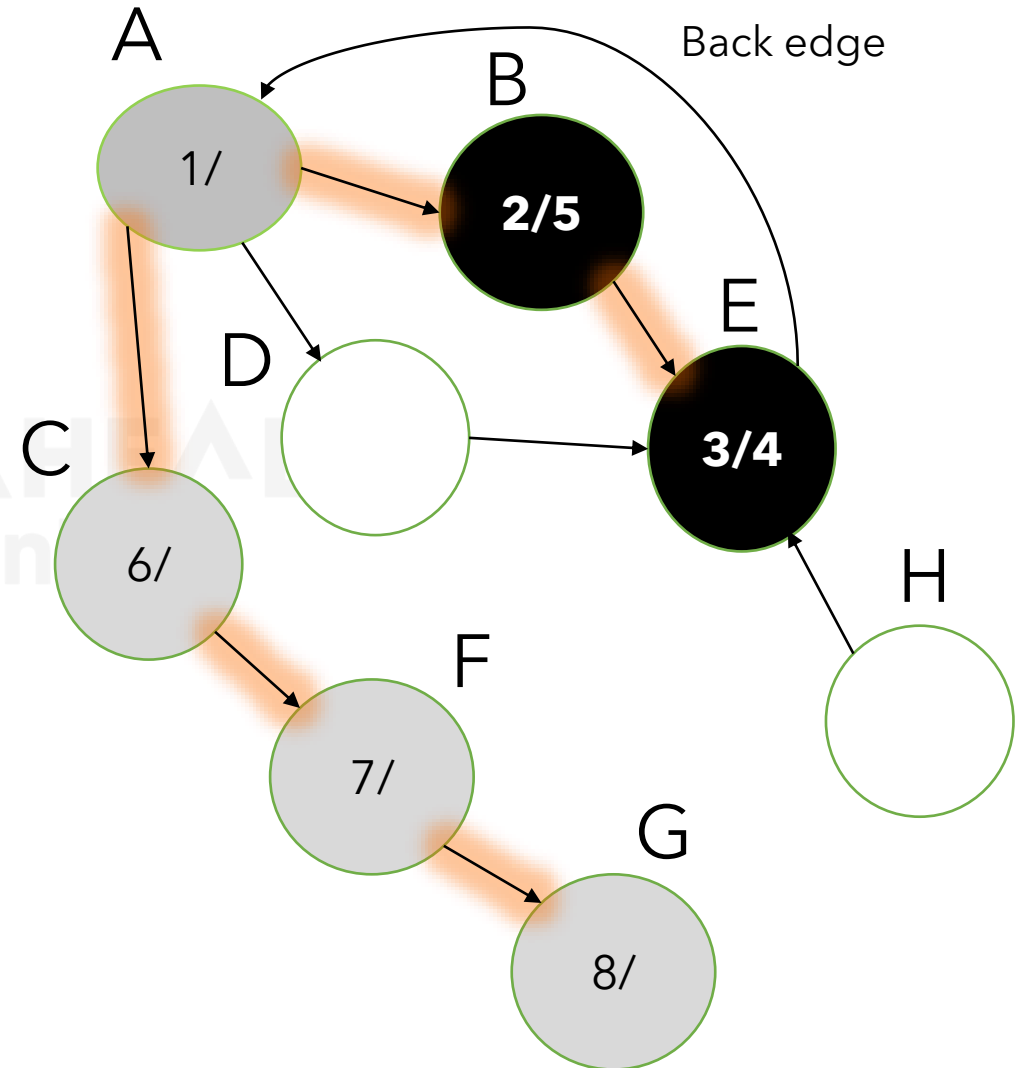


Depth First Search

T = 6



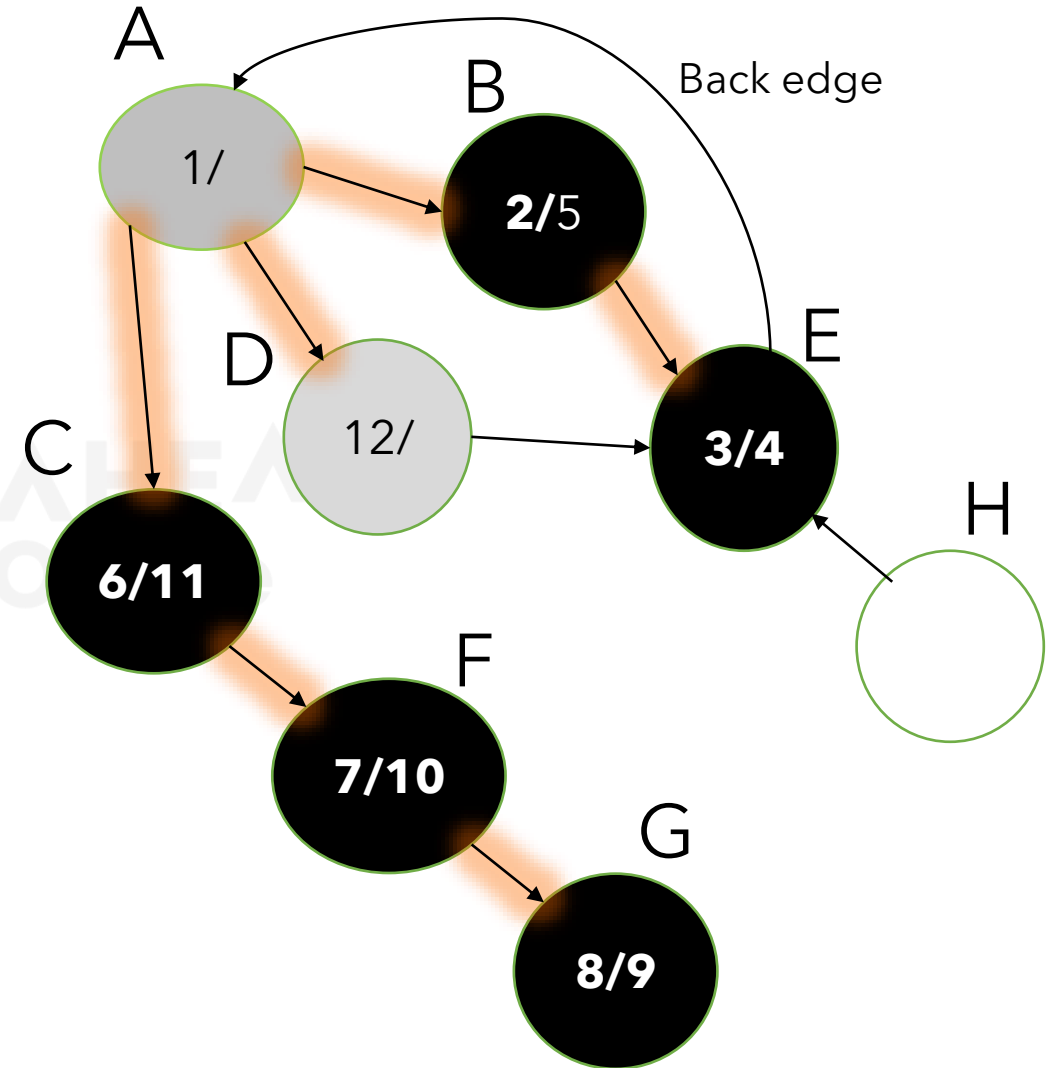
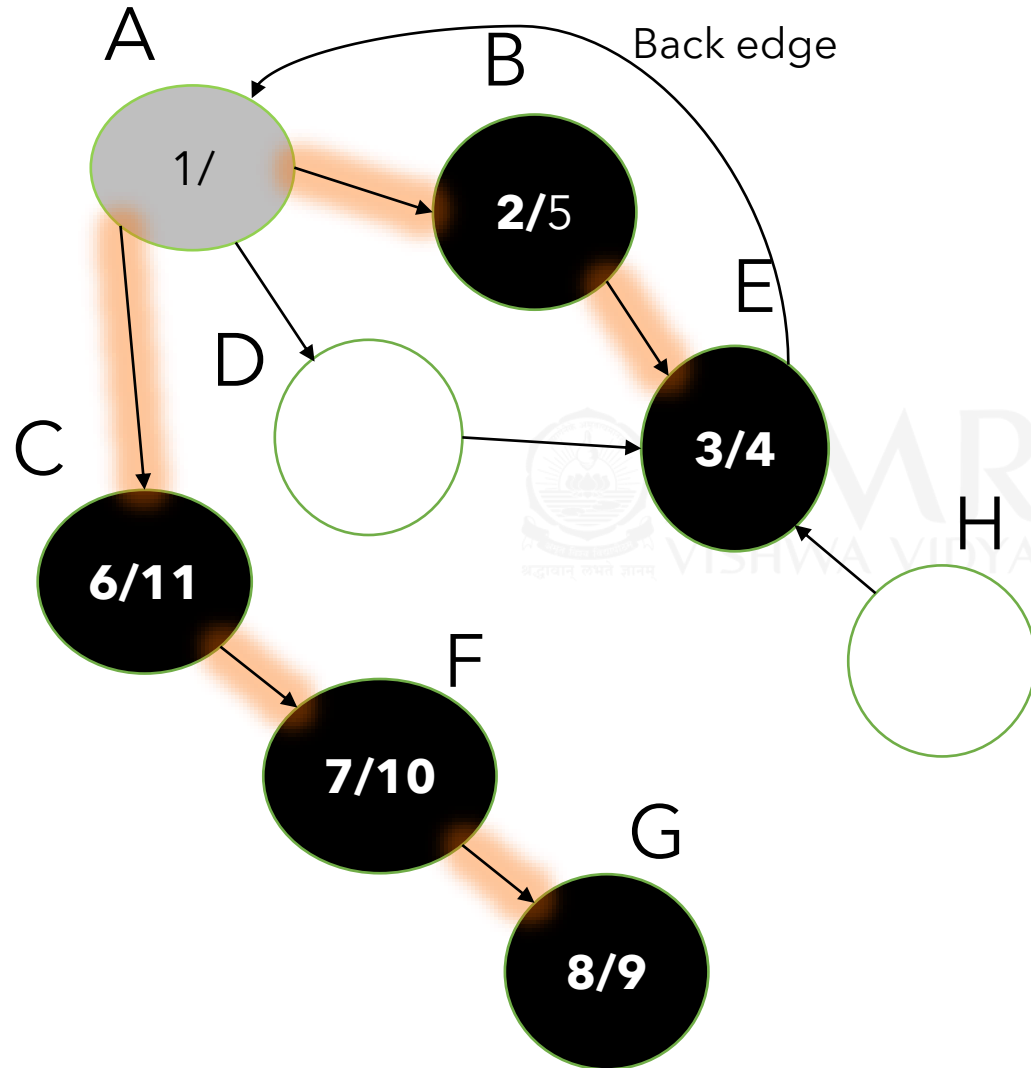
T = 7,8



$T = 9, 10, 11$

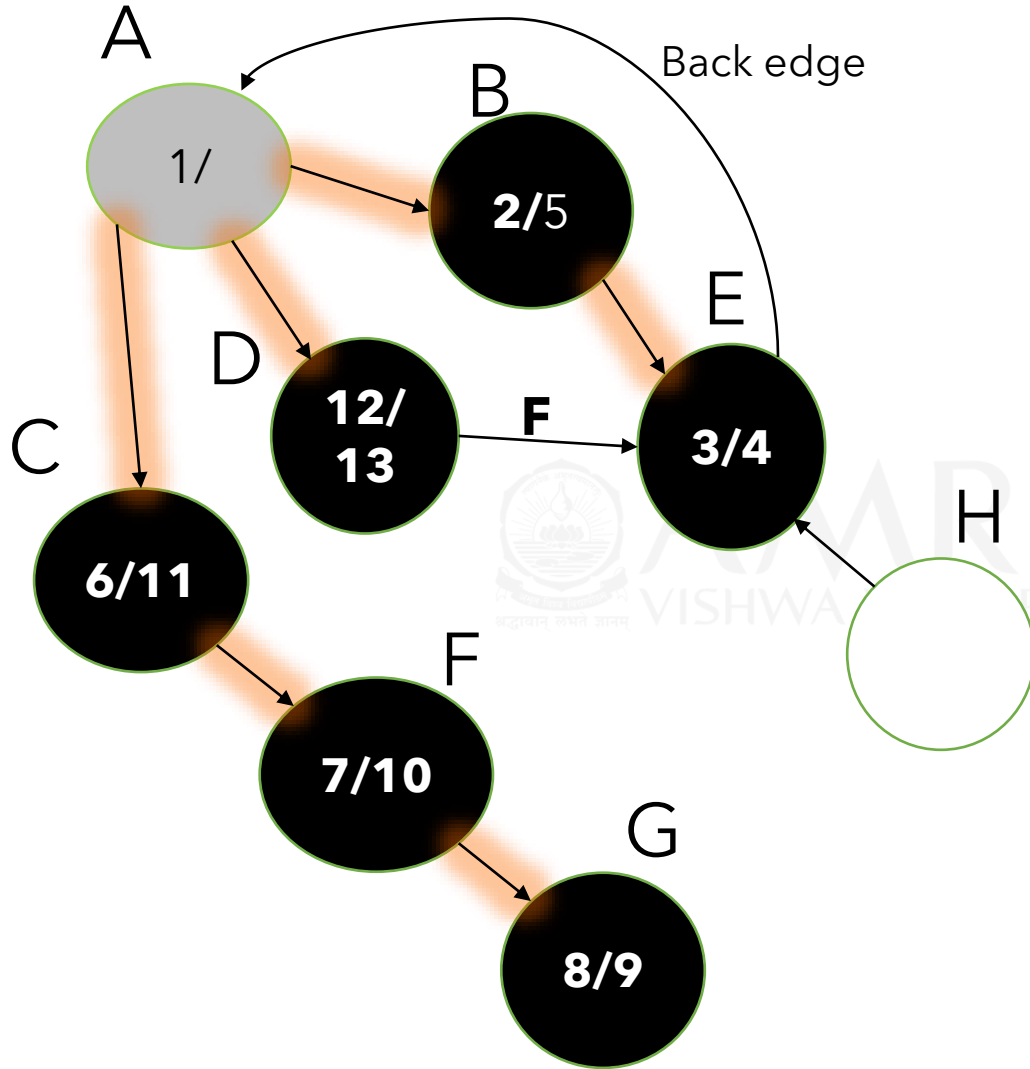
Depth First Search

$T = 12$

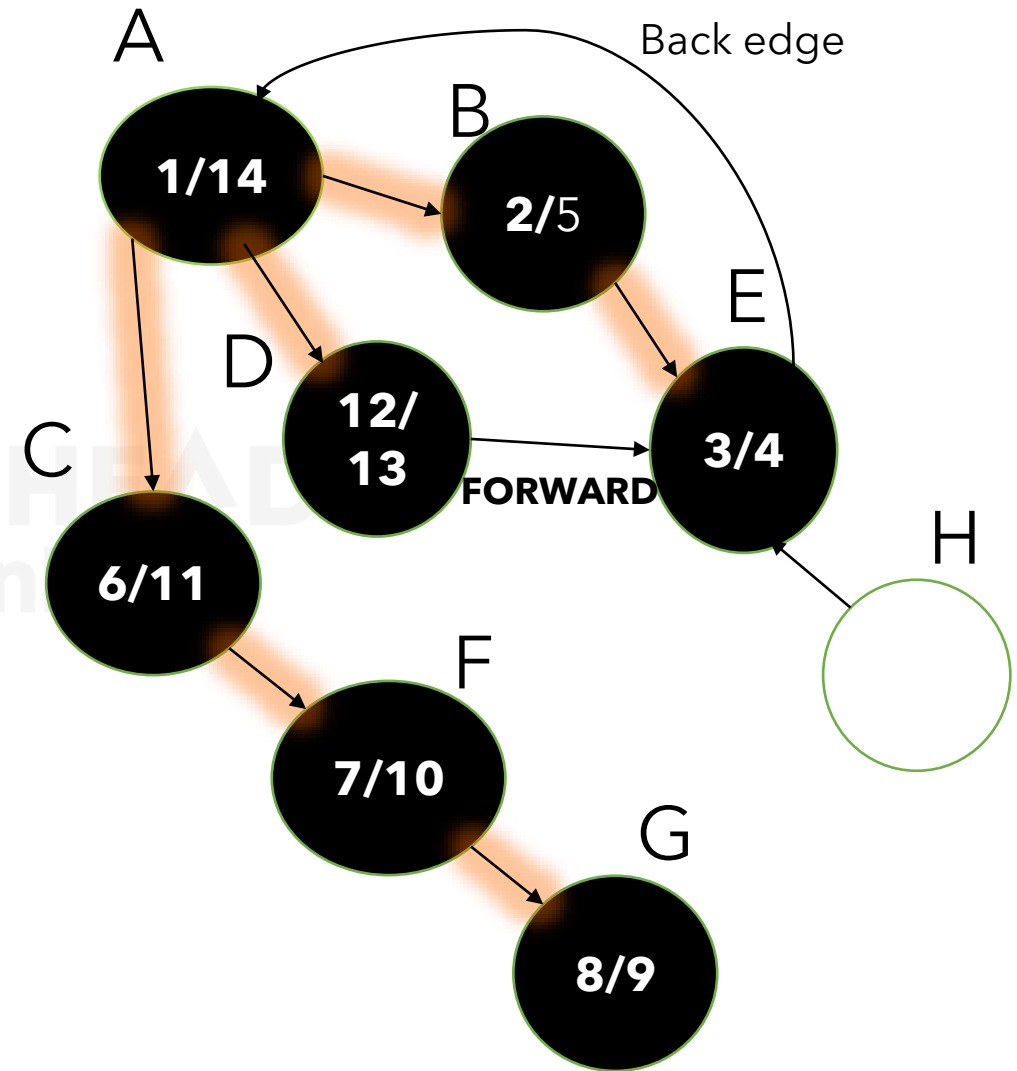


Depth First Search

$T = 13$

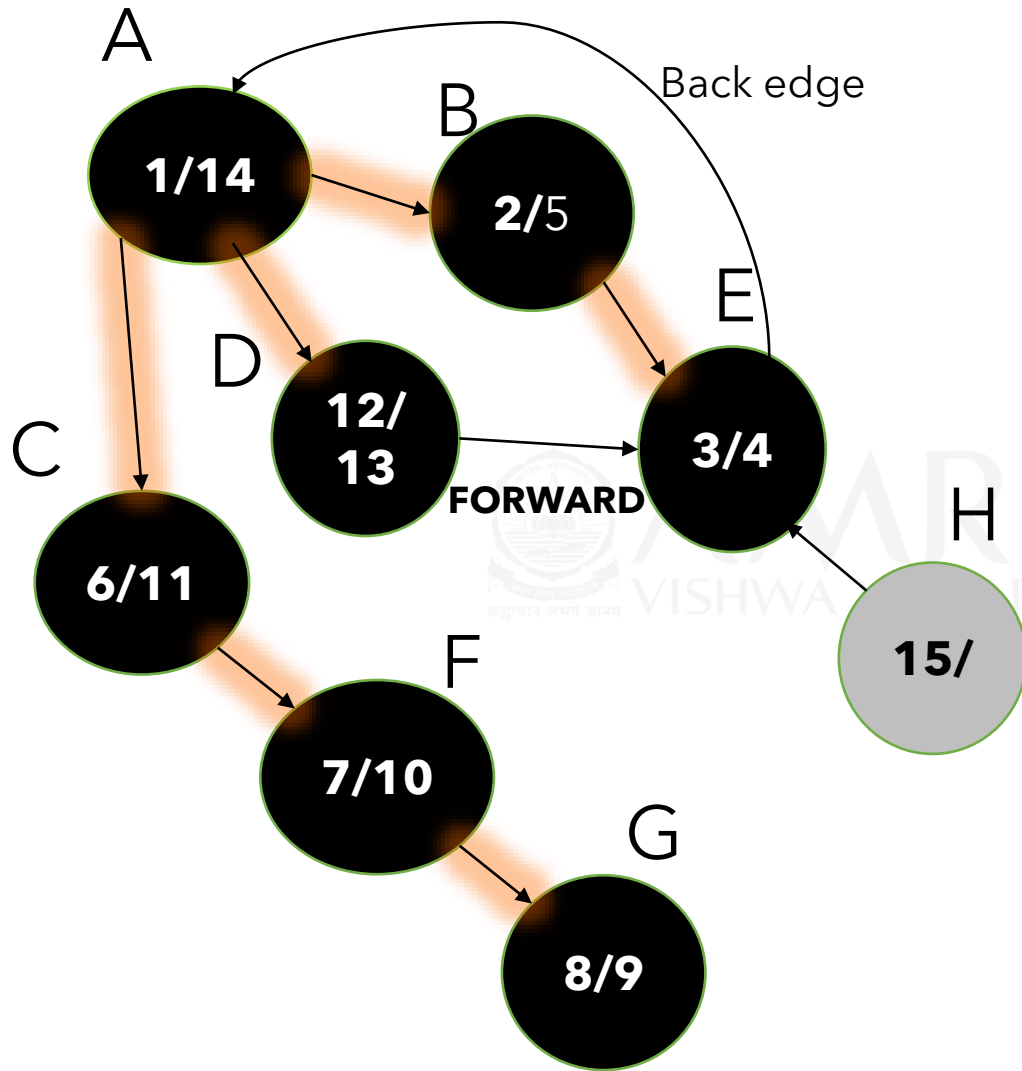


$T = 14$

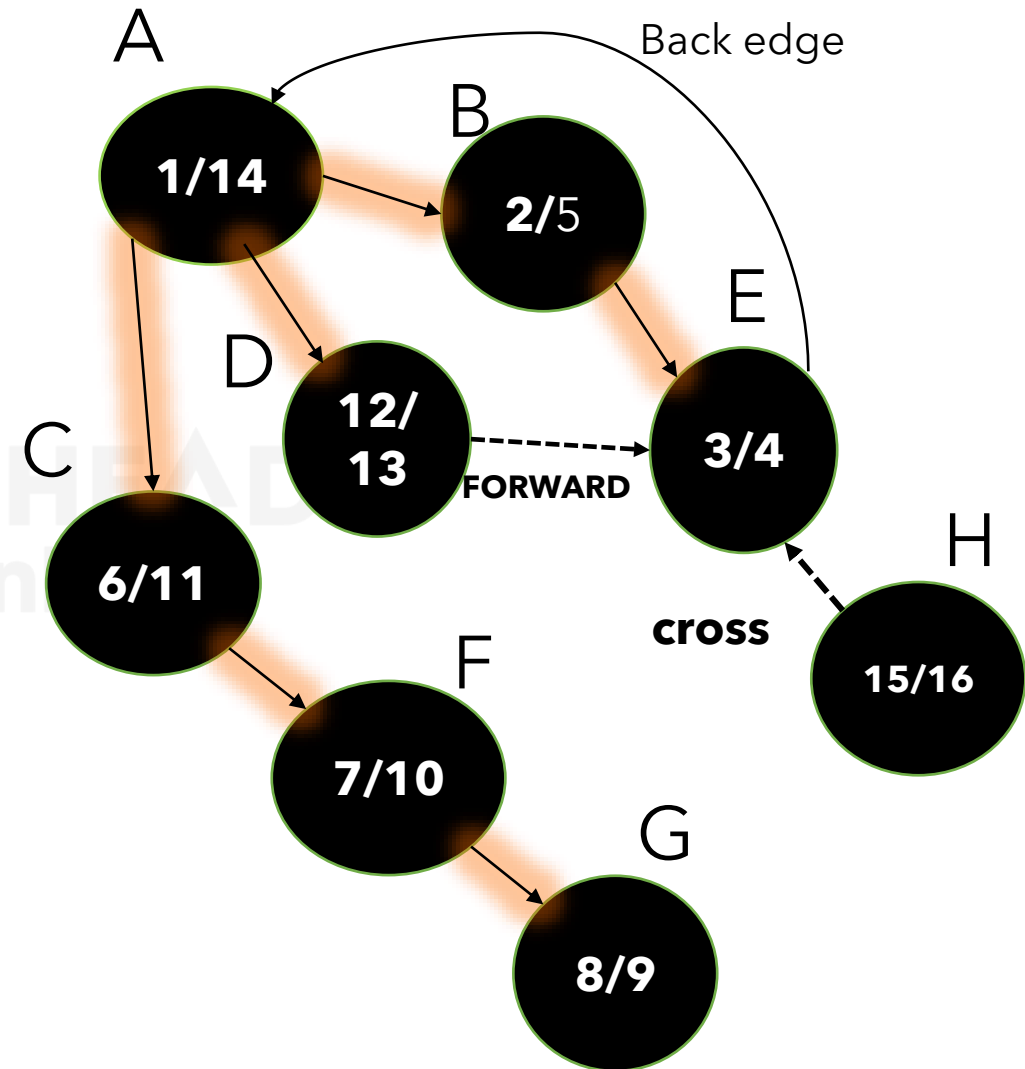


Depth First Search

T = 15



T = 16



Depth First Search

```
1 DFS(G):  
2   for each vertex  $u \in G.V$  do  
3      $u.color = WHITE$   
4      $u.pred = NIL$   
5   end  
6    $time = 0$   
7   for each vertex  $u \in G.V$  do  
8     if  $u.color == WHITE$  then  
9       DFS-VISIT(G,u)  
10    end  
11  end  
12 end
```

```
1 DFS-VISIT(G, u):  
2    $time = time + 1$   
3    $u.d = time$   
4    $u.color = GRAY$   
5   for each vertex  $v \in G.Adj[u]$  do  
6     if  $v.color == WHITE$  then  
7        $v.pred = u$   
8       DFS-VISIT(G,u)  
9     end  
10     $u.color = BLACK$   
11     $time = time + 1$   
12     $u.f = time$   
13  end  
14 end
```

Depth First Search - Properties

- Predecessor subgraph forms a forest of trees.
 - Vertex v is a descendent of vertex u , in the depth-first forest, if and only if v is discovered during search in u 's adjacency list, when u is grey.
 - Discovery and finishing time has **parenthesis structure**.
- | | | | | | | | | | | | | | | | |
|------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| • 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| • (A | (B | (E | (E | B) | (C | (F | (G | (G | F) | C) | (D | (D | A) | (H | H) |

Depth First Search - Properties

- Discovery and finishing time has **parenthesis structure**.

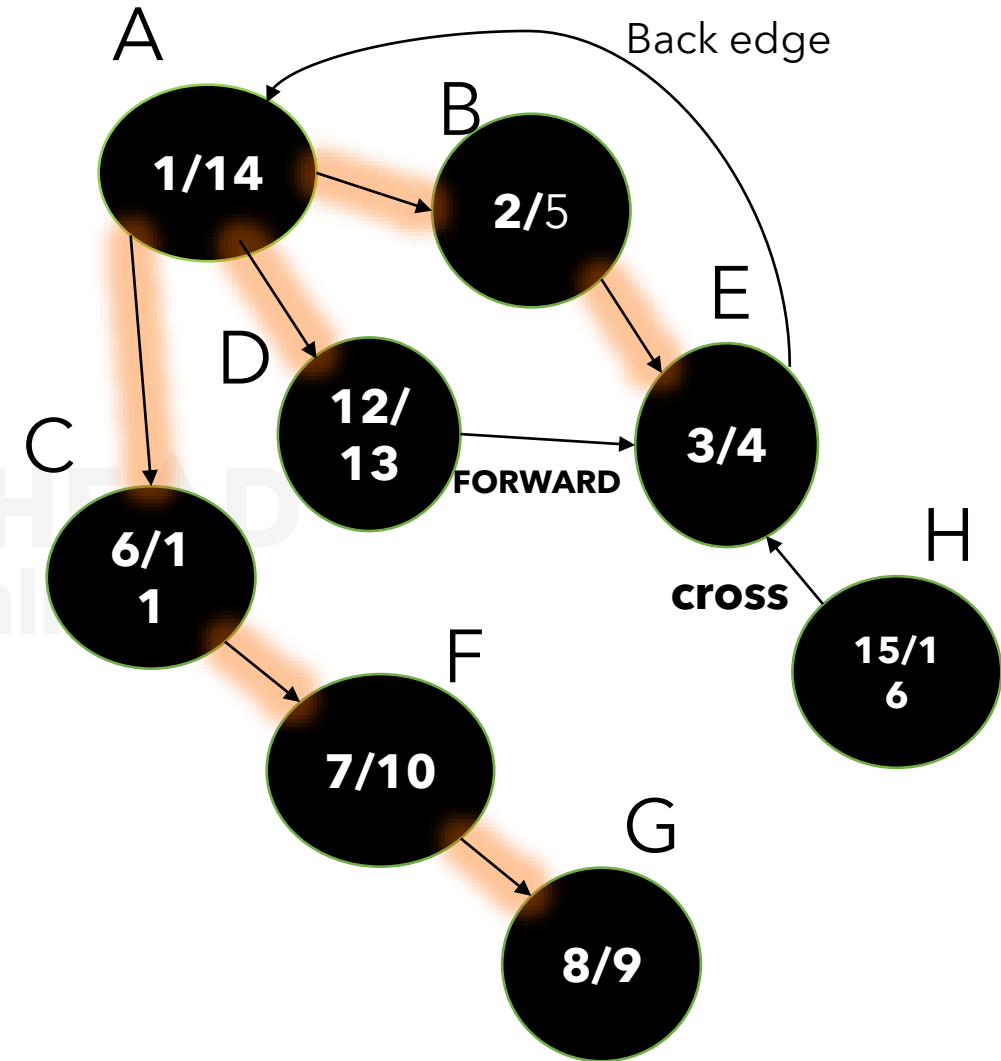
- 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
- (A (B (E (E) B) (C (F (G (G) F) C) (D D) A) (H H)



Depth First Search - Properties

Classification of edges in the graph

- 4 edge types
- Tree edges - edges in depth-first forest.
- Back edges - edge (u,v) connecting vertex u to an ancestor v in a depth-first tree.
- Forward edges - nontree edges (u,v) connecting a vertex u to a descendant v , in a depth-first tree.
- Cross edges are all other edges.

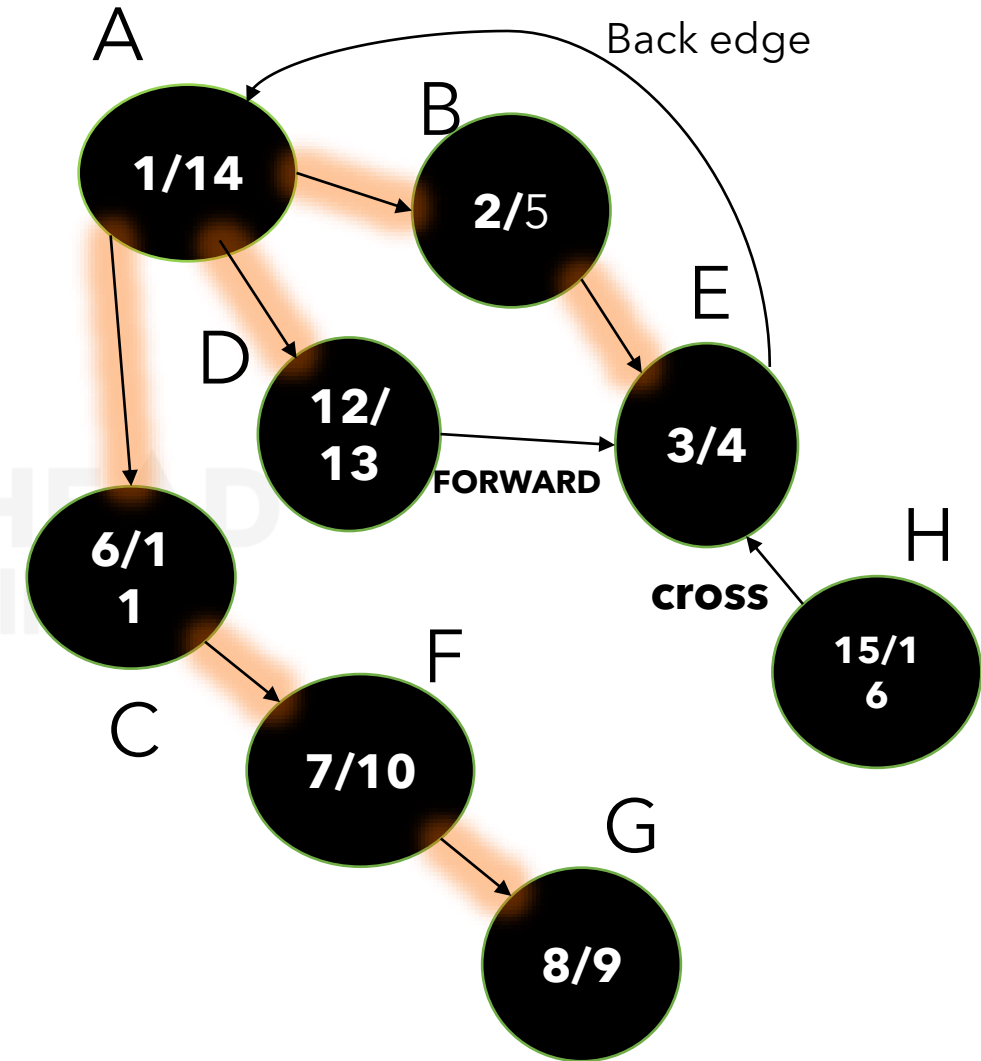


Depth First Search - Applications

Topological Sort

Finishing times in descending order

16	14	13	11	10	9	5	4
H	A	D	C	F	G	B	E



Depth First Search - Applications

- Strongly Connected Components - every vertex is reachable from every other vertex.
- Cycle in a graph - presence of backedge.



Summary

Depth-First search and its applications



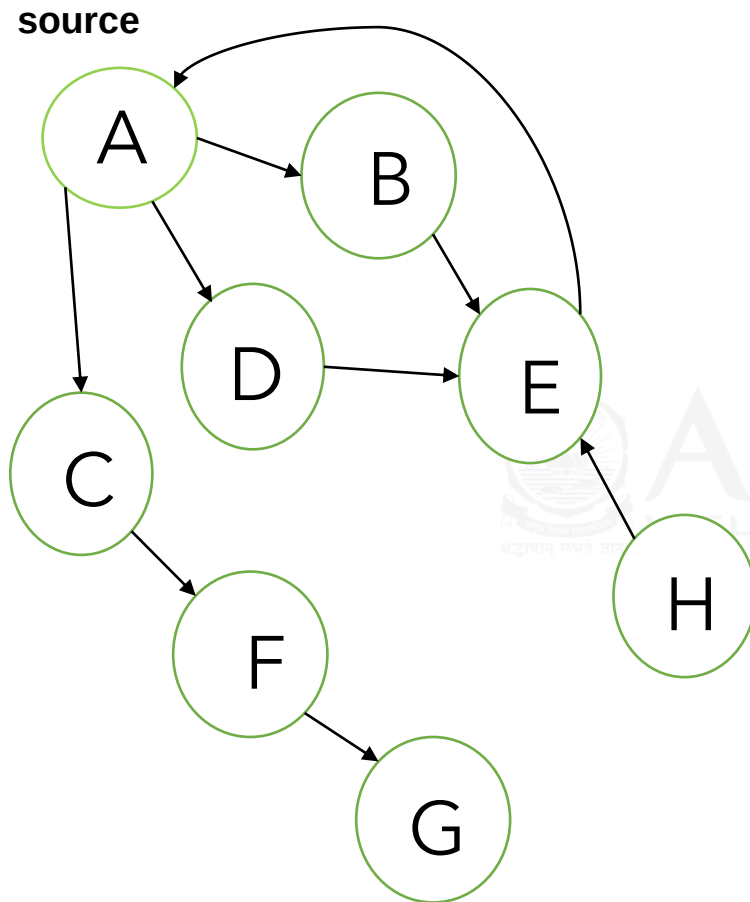
Objectives

Graph – Traversal

Depth First Search

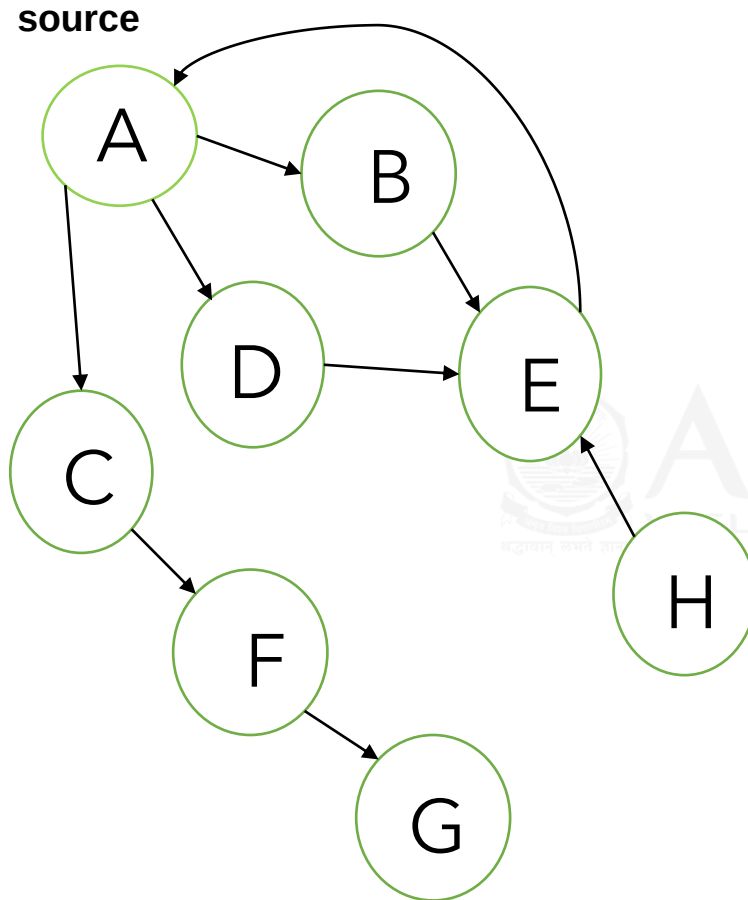


Depth First Search



- Search deeper in the graph.
- Colours vertices to indicate its state.
 - ✓ White - initially
 - ✓ Grey - discovered
 - ✓ Black - finished processing
- Each node, v has a two time stamps
 - ✓ $v.d$ = time when v is discovered.
 - ✓ $v.f$ = time when search finishes examining v 's adjacency list.

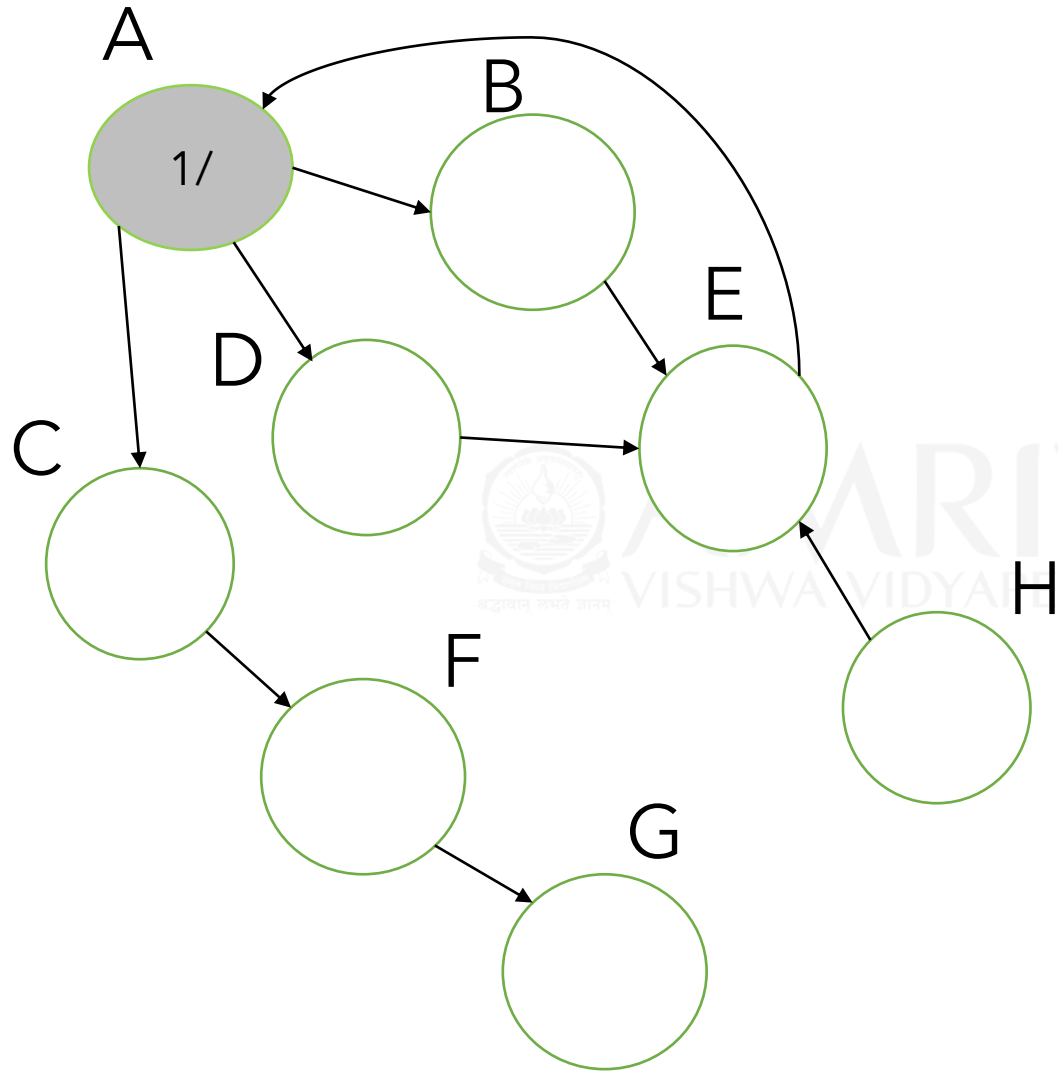
Depth First Search



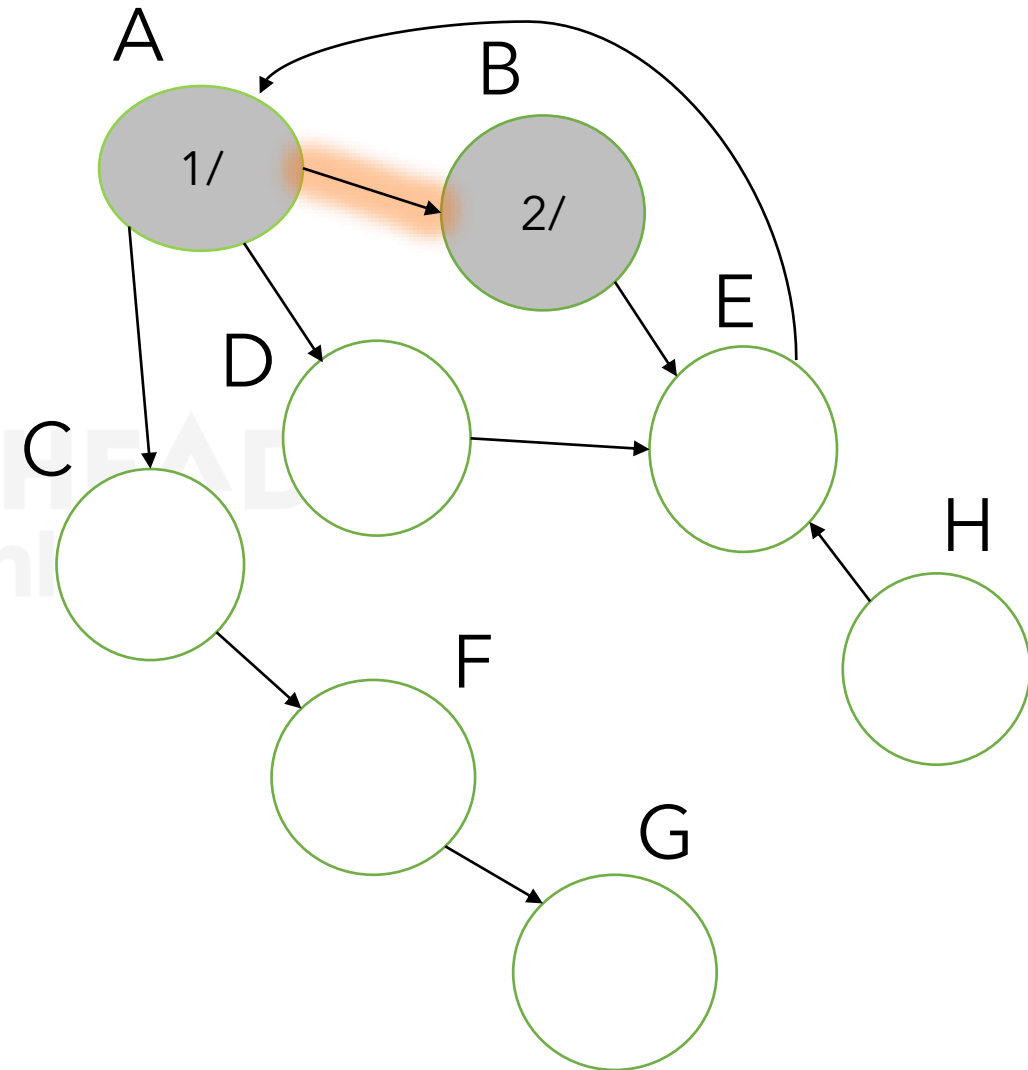
- Predecessor attribute of a node v , is set to u , if v is discovered by scanning vertices of u .
- Predecessor subgraph - depth first forest with depth first trees.

Depth First Search

T = 1

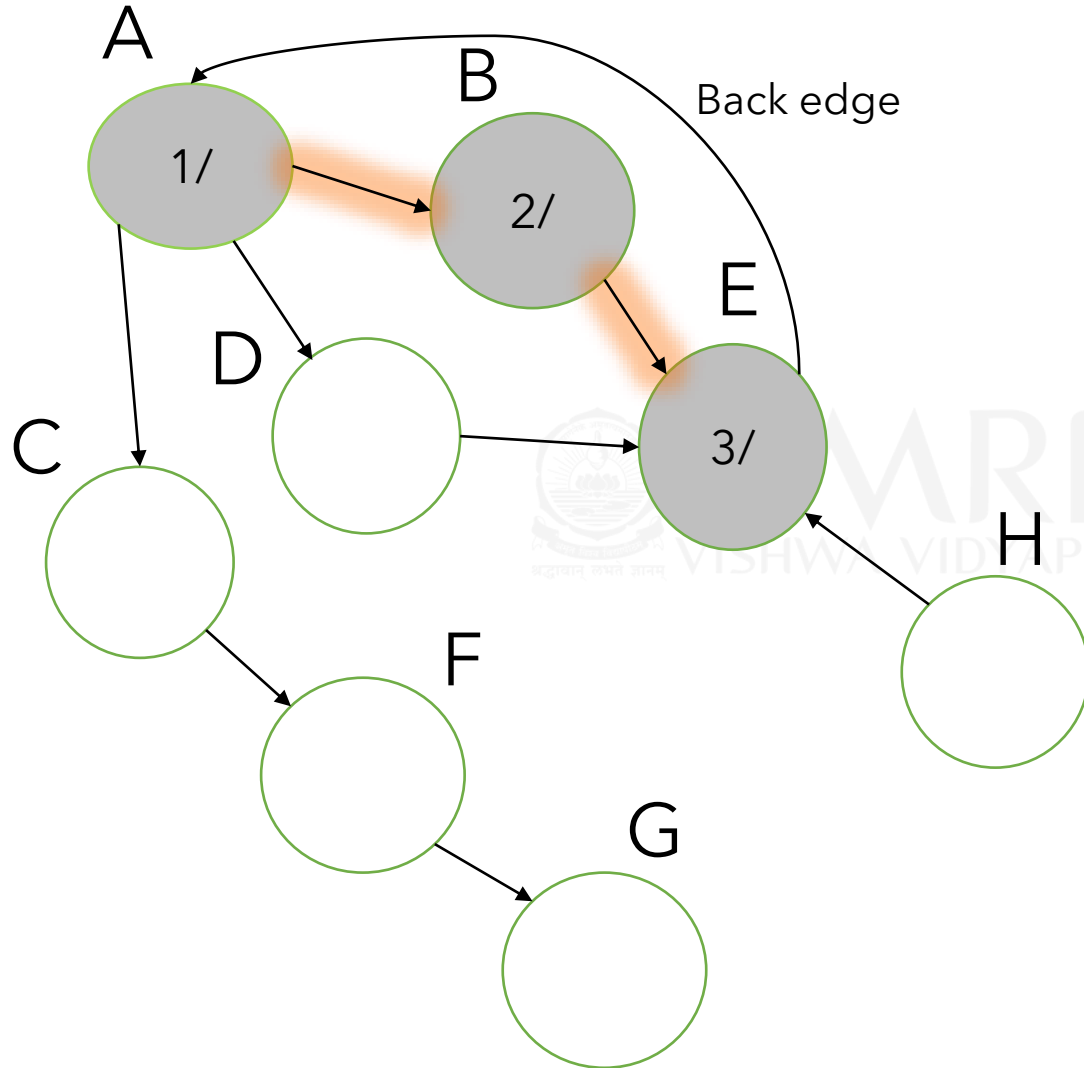


T = 2

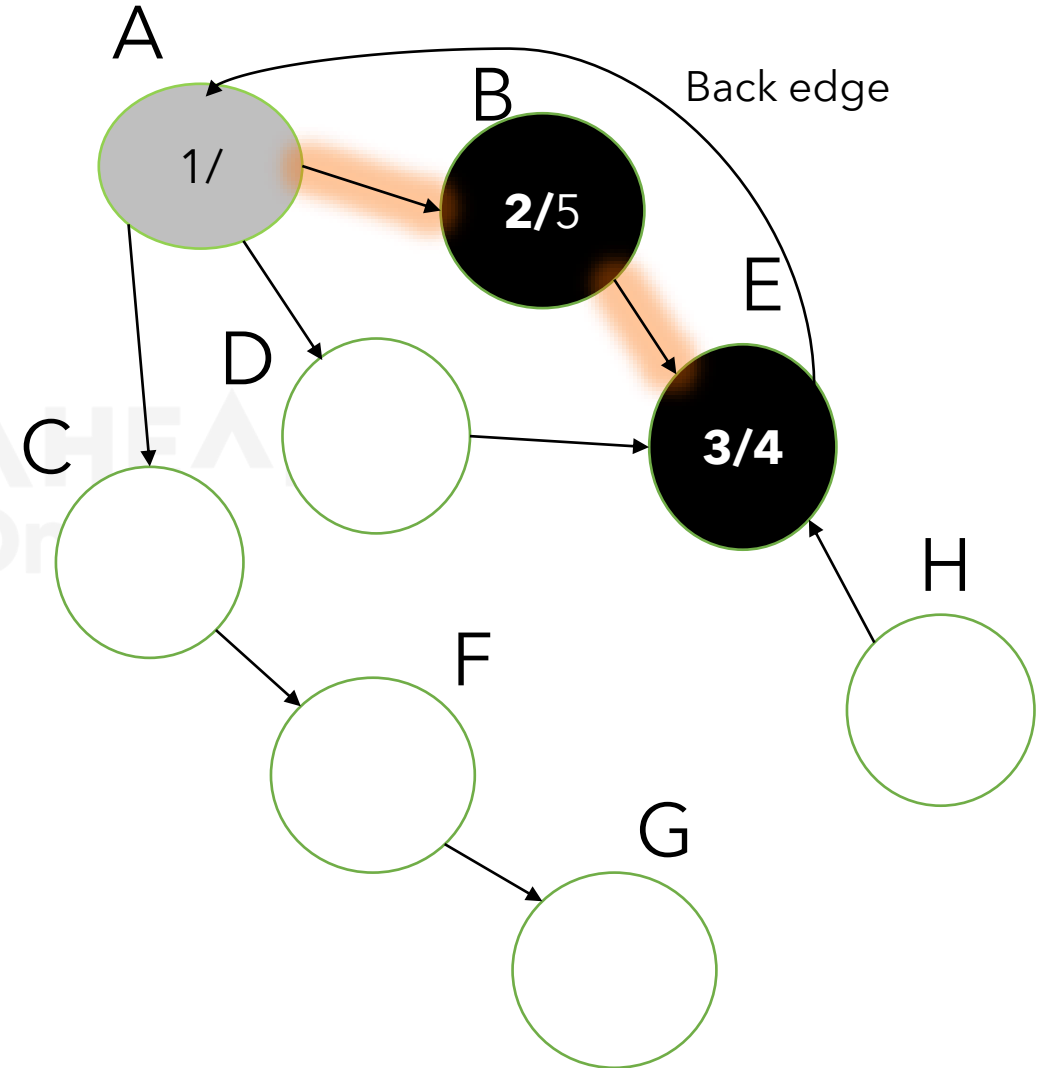


Depth First Search

T = 3

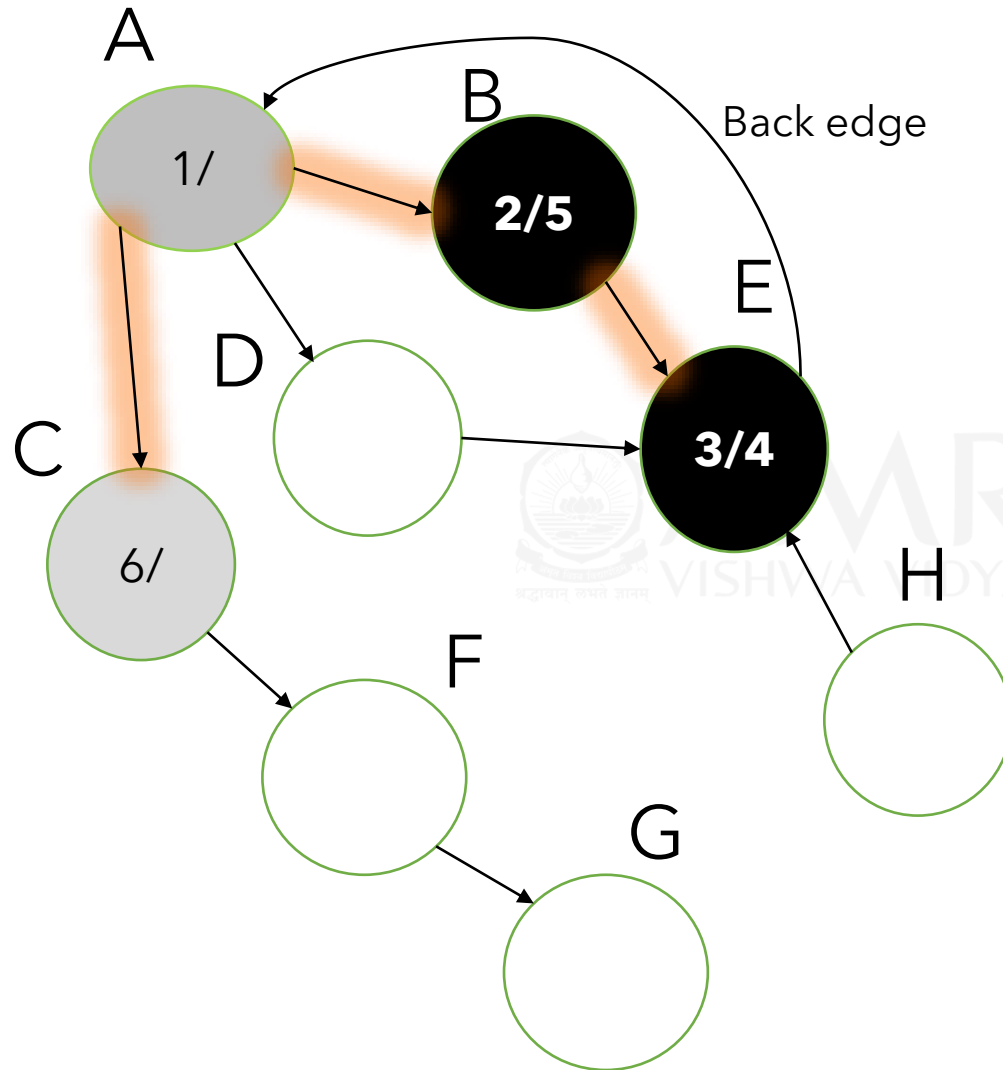


T = 4, 5

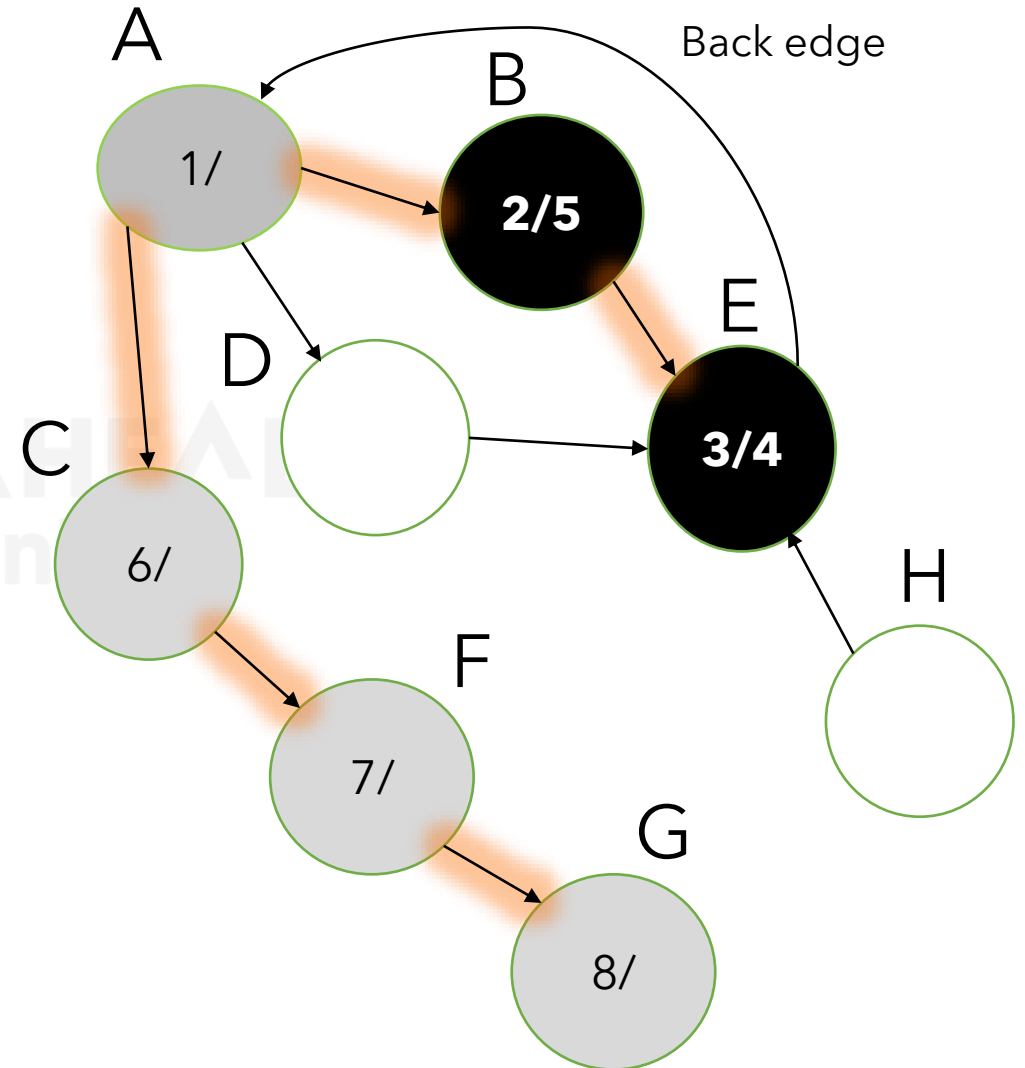


Depth First Search

T = 6



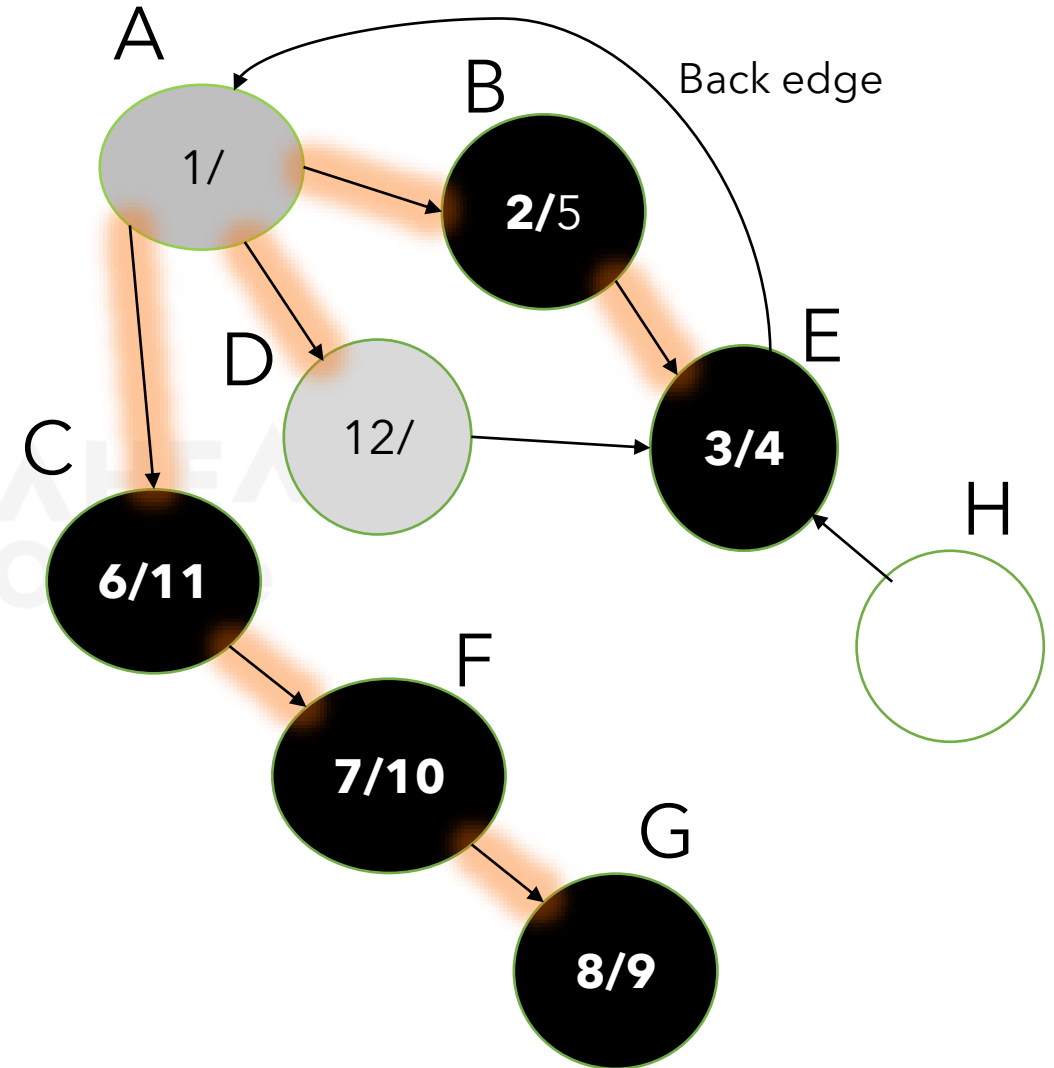
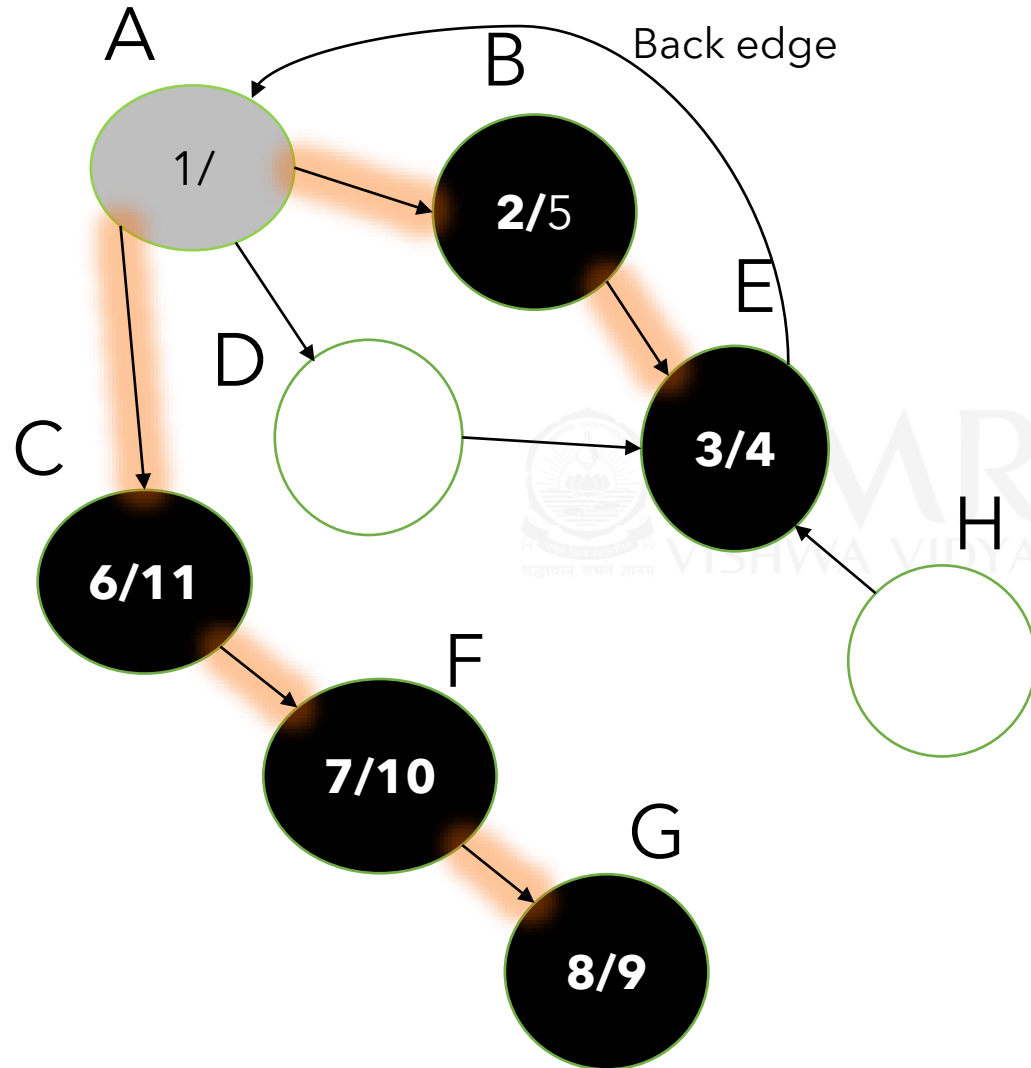
T = 7,8



$T = 9, 10, 11$

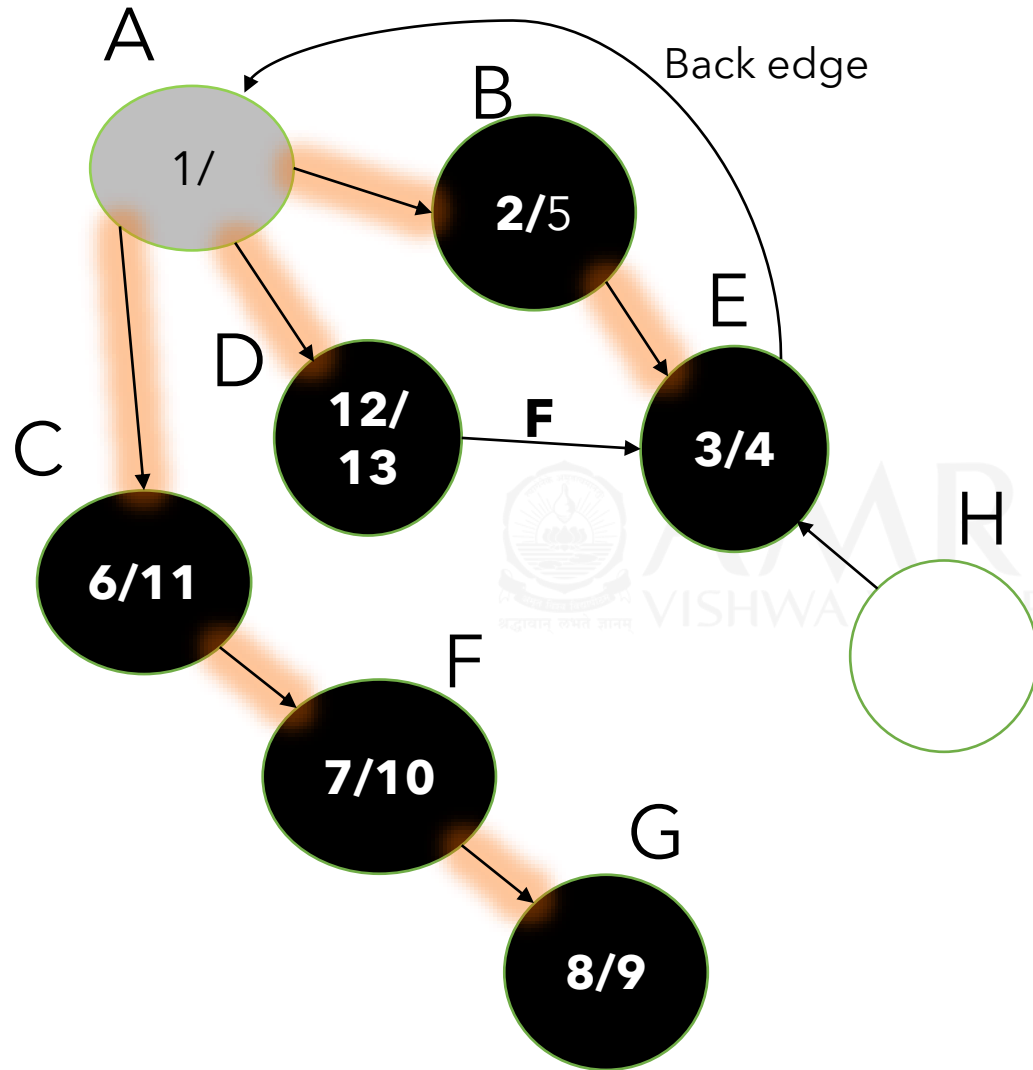
Depth First Search

$T = 12$

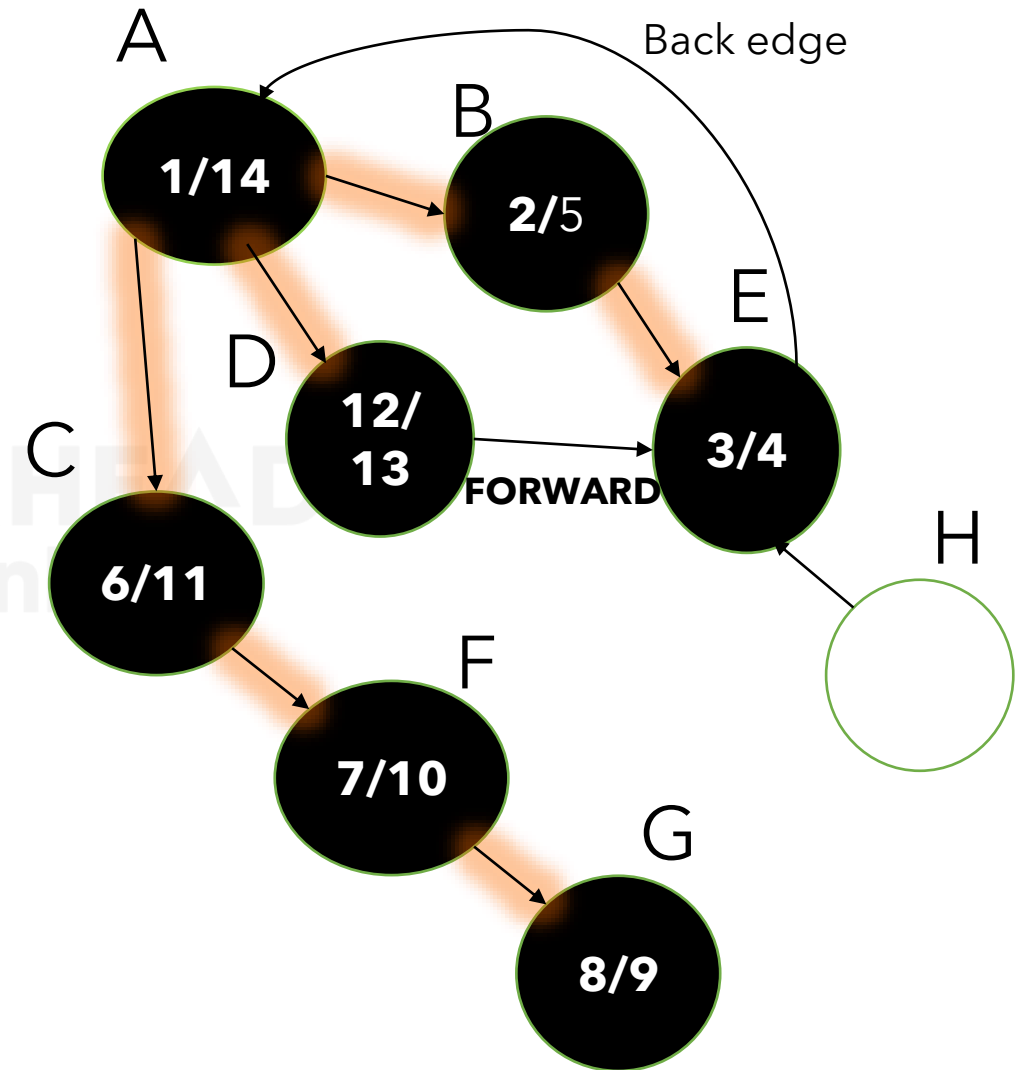


Depth First Search

$T = 13$

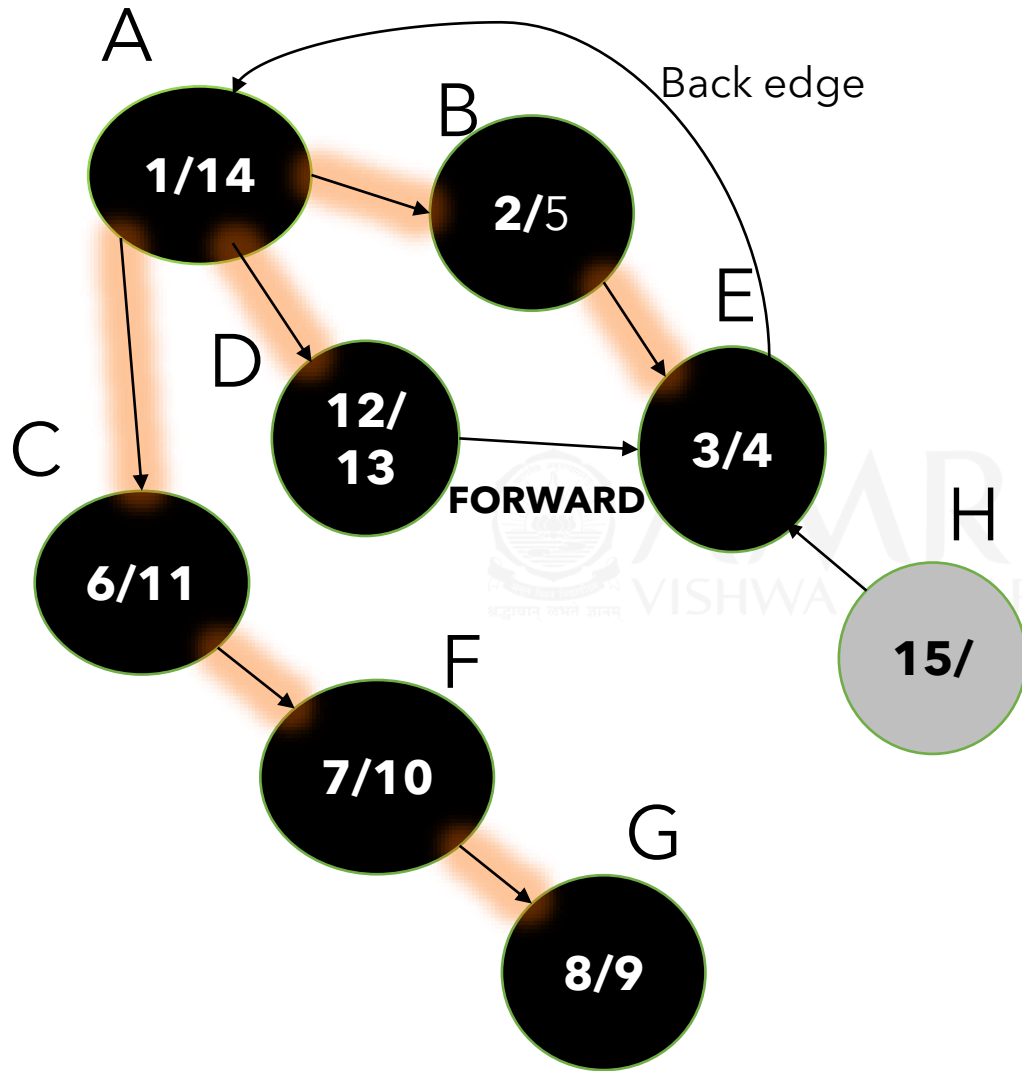


$T = 14$

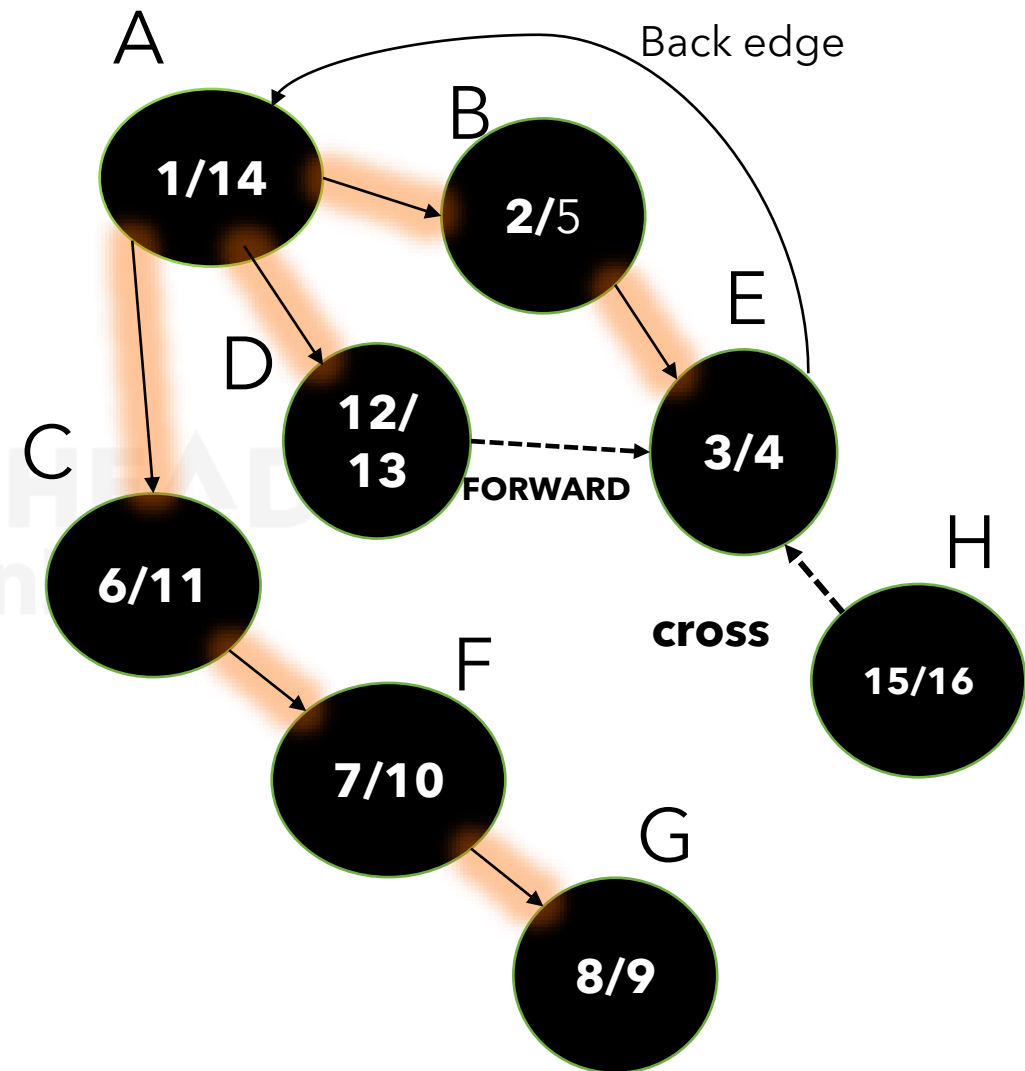


Depth First Search

T = 15



T = 16



Depth First Search

```
1 DFS(G):
2   for each vertex  $u \in G.V$  do
3      $u.color = WHITE$ 
4      $u.pred = NIL$ 
5   end
6    $time = 0$ 
7   for each vertex  $u \in G.V$  do
8     if  $u.color == WHITE$  then
9       DFS-VISIT(G,u)
10    end
11  end
12 end
```

```
1 DFS-VISIT(G, u):
2    $time = time + 1$ 
3    $u.d = time$ 
4    $u.color = GRAY$ 
5   for each vertex  $v \in G.Adj[u]$  do
6     if  $v.color == WHITE$  then
7        $v.pred = u$ 
8       DFS-VISIT(G,u)
9     end
10     $u.color = BLACK$ 
11     $time = time + 1$ 
12     $u.f = time$ 
13  end
14 end
```

Depth First Search - Properties

- Predecessor subgraph forms a forest of trees.
 - Vertex v is a descendent of vertex u , in the depth-first forest, if and only if v is discovered during search in u 's adjacency list, when u is grey.
 - Discovery and finishing time has **parenthesis structure**.
- | | | | | | | | | | | | | | | | |
|------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| • 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
| • (A | (B | (E | (E | B) | (C | (F | (G | (G | F) | C) | (D | (D | A) | (H | H) |

Depth First Search - Properties

- Discovery and finishing time has **parenthesis structure**.

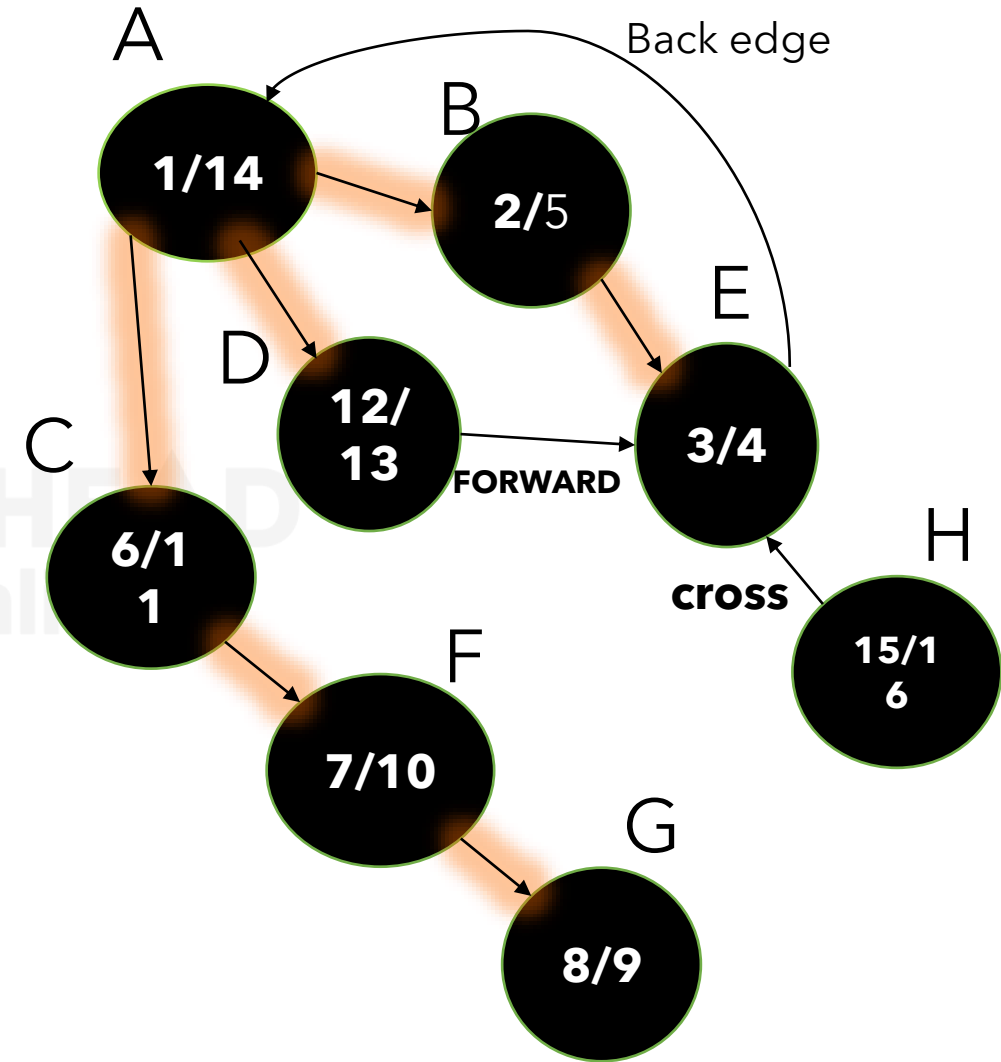
- 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
- (A (B (E (E B) (C (F (G G) F) C) (D D) A) (H H)



Depth First Search - Properties

Classification of edges in the graph

- 4 edge types
- Tree edges - edges in depth-first forest.
- Back edges - edge (u,v) connecting vertex u to an ancestor v in a depth-first tree.
- Forward edges - nontree edges (u,v) connecting a vertex u to a descendant v , in a depth-first tree.
- Cross edges are all other edges.

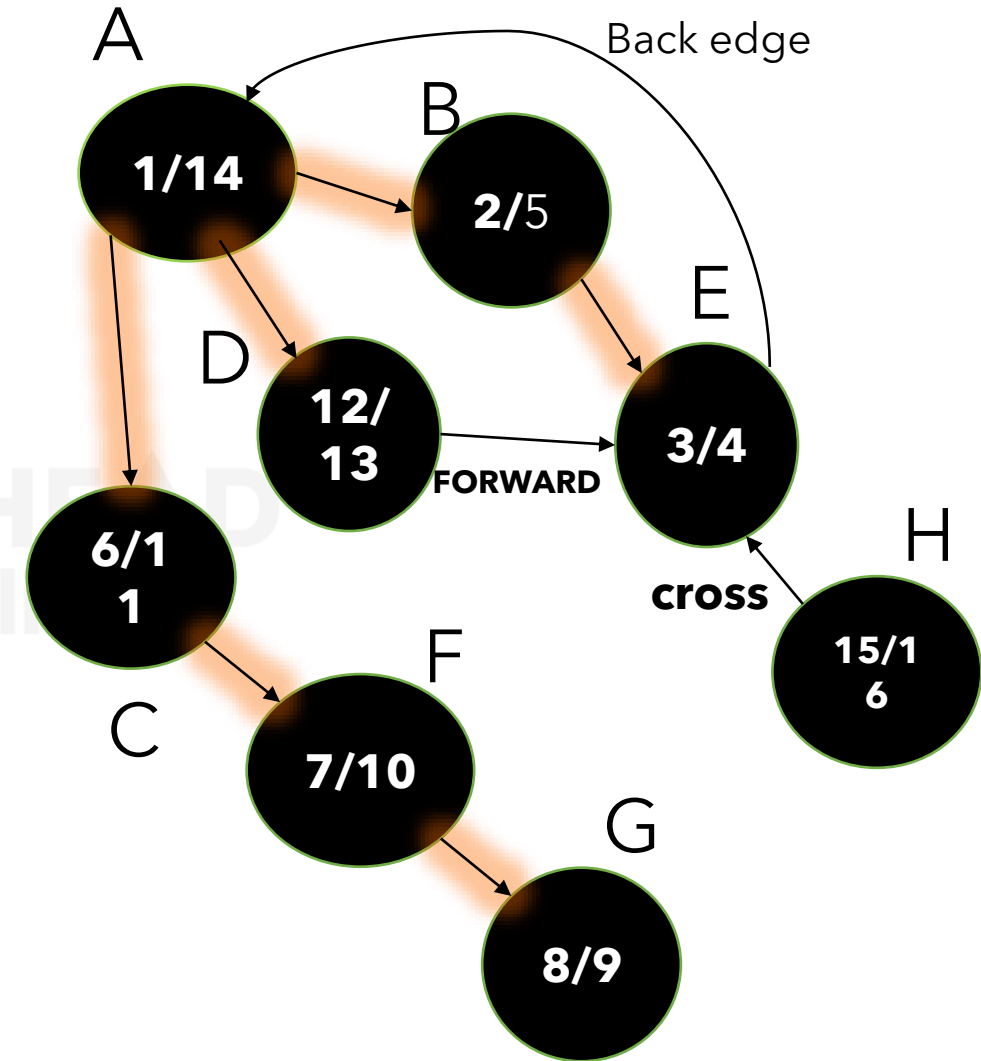


Depth First Search - Applications

Topological Sort

Finishing times in descending order

16	14	13	11	10	9	5	4
H	A	D	C	F	G	B	E



Depth First Search - Applications

- Strongly Connected Components - every vertex is reachable from every other vertex.
- Cycle in a graph - presence of backedge.



Summary

Depth-First search and its applications

